

Approximating Graph Inference by Serving Graph Model Outputs as Views

Yangxin Fan, Mingjian Lu, Haolai Che, Yinghui Wu

Case Western Reserve University

Cleveland, Ohio, USA

{yxf451, mxl1171, hxc859, yxw1650}@case.edu

Abstract—Graph machine learning models are increasingly served as analytical operators, yet repeated inference over large graphs remains expensive. We study graph inference serving as a data-management problem: how to materialize, select, minimize, and reuse historical graph-model computations as views. Given a fixed graph model M , a collection of memory graphs, and historical executions of M , a graph model view (GMV) stores a memory graph, layer-aware model representations, and auxiliary matching and message-reconstruction metadata. For an inference query, the graph owner locally compares the bounded computation neighborhood with candidate memory graphs to derive match relations and model-specific reconstruction coefficients. The GMV service then selects and assembles relevant views to reconstruct an exact or approximate query result without rerunning the complete inference process over the original graph. We establish sufficient conditions for exact reconstruction and metadata-computable error bounds for approximate inference, and develop algorithms for view-based inference, view selection, view minimization, and workload-oriented serving. Experiments on benchmark graphs show speedups of up to $850\times$ with small accuracy loss, scalability on billion-scale synthetic graphs, and practical applications to cybersecurity and Bitcoin transaction anomaly detection.

I. INTRODUCTION

Graph (machine learning) models, such as graph neural networks (GNNs) [1] have been routinely queried to support graph analysis. A graph model M converts an input representation of a graph $G = (X, A)$, where X (resp. A) is a matrix of node features (resp. adjacency matrix) of G , to a numerical matrix Z (“logits”) via an inference process. Given a graph G , a test node v_t in G , and a GNN M , an “inference query” $Q(M, v_t, G)$ applies the inference process of M over G to return user-specific output $Z(v_t)$ (or Z by default). The output Z can be post-processed for downstream analysis, such as labels for classification, or numerical values for regression analysis. A query workload can be posed to retrieve the outputs $Z(V_T)$ for a set of test nodes V_T , as v_t ranges over V_T in G .

We treat this workload as an inference-serving problem, rather than as a request to retrain or redesign M . The central data-engineering question is: whether previous graph-model computations can be “materialized” as reusable views and subsequently selected, transformed, and assembled to answer new inference queries with lower online cost.

Example 1: Fig. 1 illustrates a fragment of a molecular graph G that encodes a new drug under development. Nodes in G represent atoms and edges represent chemical bonds. It

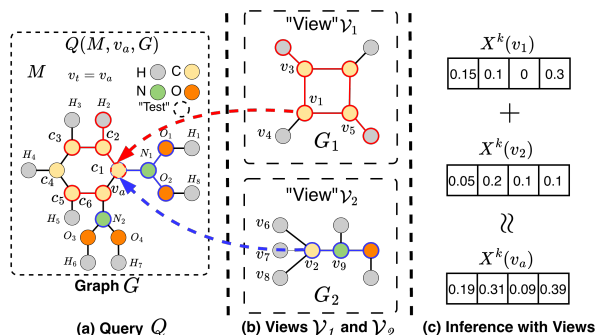


Fig. 1: Approximate Evaluation of an Inference Query Q over Target Molecular Graph G Using “Views” $\mathcal{V}_1$ and $\mathcal{V}_2$.

contains $C_6H_8N_2O_4$ (a benzene ring with two hydrogens substituted by functional groups), a useful compound to understand electronic, structural, or reactivity-related properties in computational chemistry [2]. However, the presence of its chemically unstable structures such as reactive hydroxylamine groups ($-NOH$) make in-lab tests expensive to verify properties such as polarizability under various settings [3].

A chemical data scientist wants to exploit graph models, *e.g.*, a vanilla GNN M [4]. She may issue an inference query $Q(M, v_a, G)$ on a designated atom v_a associated with $-NOH$ group to understand how its local chemical structure contributes to predict molecular properties, such as toxicity, solubility, polar surface area, and polarizability, as 4-ary numeric vectors (normalized in $[0, 1]$). By performing node-level inference on v_a , the chemist can estimate the effects of functional groups on the properties. Such output is obtained by an iterative message passing over G using an update rule $X_v^k = \sigma(\Theta \cdot \text{AGG}(X_u^{k-1}, \forall u \in \mathcal{N}(v)))$, that requires atom-level features X and an adjacency matrix A of G . After L layers of propagation, M outputs four-dimensional vectors (embeddings) $Z(v)$ at each node v , representing the predicted contribution of atom v to the overall molecular properties. The output $Z(v)$ will be post-processed to derive the aforementioned molecular properties. $\square$

While the need is evident, a straightforward inference over G remains expensive or infeasible due to access control [5], [6], or when G is large (*e.g.*, a large design space of combinations of compounds). In the above example, data providers may not reveal the entire G to queryers. On the other hand, they may post public data samples at data hub services such as

Hugging Face [7], which resemble sufficient “similar” counterpart in G to provide an “approximated” answer to queryers. A natural interest is to best reuse such results as public “views”, to approximate $Q(M, V_T, G)$ as fast and accurate as possible, supported by e.g., federated data infrastructures [8].

Example 2: Consider the inference request posed by the chemical scientist. While a direct access to G may undergo long process, metadata is made available on structurally and functionally similar molecules. For example, the chemical properties of Cyclobutadiene (C_4H_4 , represented by molecular graph G_1) and methylhydroxylamine (CH_5NO , represented by G_2) are publicly available in chemical databases at ChemSpider [9] and NIST WebBook [10].

Is it possible to approximately answer Q by consulting the model output over G_1 and G_2 ? While neither is strictly a subgraph of G , they exhibit functional and topological similarities that contribute to almost indistinguishable roles in an inference process given their similar counterpart in G (e.g., benzene ring-NOH groups). Indeed, as justified by fragment modeling in computational chemistry, such structurally similar fragments often demonstrate similar electronic and structural behavior that contribute to chemical properties [11].

We consider wrapping inference outputs over G_1 and G_2 as “views” that can be served, shared, and reused by queryers over a federated memory infrastructure. Without requesting an inference of M that require the entire G from scratch, the queryer may request an on-site test that derives a “matching” from public G_1 and G_2 to their counterparts that participated inference process in G , such that useful metadata can be served to approximately reconstruct the output, without revealing G . For example, from the “matching” of v_1 and v_2 to v_a , we can rewrite the k -th layer embedding $m_{v_a}^k$ as a message $\hat{m}_{v_a}^k$:

$$\begin{aligned} m_{v_a}^k &\approx \hat{m}_{v_a}^k = \sum_{i \in \{1,2\}} \sum_{u' \in N_{G_i}(v_i)} s_{i,u'}^k X_{u'}^{k-1}, \\ X_{v_a}^k &\approx \hat{X}_{v_a}^k = \sigma(\Theta^k \hat{m}_{v_a}^k). \end{aligned}$$

where X_v^{k-1} (resp. X_v^k) is the embedding of a node v at the $(k-1)$ -th (resp. k -th) layer; σ is an activation function, $N_{G_i}(v)$ refers to the neighbors of a node v in a graph G_i ($i \in [1,2]$), and Θ refers to the learned weight matrix (same across all layers in the GNN).

The graph owner computes the matching and correction metadata locally. The service reuses their layer inputs or stored sufficient statistics as useful metadata to estimate the message received by v_a , and then applies the target node-update function. The benzene-ring and $N(OH)_2$ views therefore contribute reusable message components¹. Such metadata (e.g., degrees, weights, hyper-parameters) can be precomputed and reused in constant time via memoization. Better still, we expect an inference time cost determined by the size of views and auxiliary metadata rather than the size of G . $\square$

¹We remark that the service does not simply add final embeddings from G_1 and G_2 ; this distinction is essential because nonlinear activation generally does not distribute over addition.

Analogizing view-based query processing [12], we call this novel strategy *view-based graph inference*. Although desirable, several technical challenges remain.

- How to decide when an inference query Q can be evaluated by such “views” without referring to the original graph G ?
- How to reduce the size of views that are needed for such evaluation, hence the computational cost?
- How to approximate original output using views to minimize inference cost with small accuracy loss?

Contribution. We make the following contributions.

(1) We introduce *graph model views* (GMVs) $\mathcal{V}$, a class of memorization structures for view-based graph inference. Likening “views” as queries and their results, a GMV $\mathcal{V}$ packages a “memory” graph $G_{\mathcal{V}}$ (a fraction of published graphs), a memoization structure $\tau_{\mathcal{V}}$ of useful metadata, and the local inference output of M over published $G_{\mathcal{V}}$. The view-based inference aims to approximately answer an inference query $Q(M, v_t, G)$ by only referring to GMVs $\mathcal{V}$.

(2) We study three fundamental problems for view-based graph inference. These analysis provide theoretical justification and algorithm components for view-based inference algorithms.

(a) *View selection.* Given $Q(M, v_t, G)$ and a set of GMVs $\mathcal{V}$, decide whether it suffices to refer to $\mathcal{V}$ alone to answer Q , and identify a minimal subset of $\mathcal{V}'$ that can best serve the need. We provide an efficient selection algorithm with optimality guarantees, quantified by a coverage property that measures how well their matches in G cover almost “indistinguishable” fraction between the memory graphs and graph G .

(b) *View minimization.* Given GMVs $\mathcal{V}$, we introduce preprocessing algorithms to obtain a minimal representation $\mathcal{V}_m$ of $\mathcal{V}$ that preserves $Q(M, v_t, G)$ as much as possible. We present two strategies. The first compresses memory graphs into a canonical form by finding nodes that are equivalent in terms of inference process; the second further prunes nodes with contribution dominated by other participating counterparts.

(c) *Inference using views.* We introduce efficient algorithms that refers to GMVs $\mathcal{V}$ to approximately answer Q , and provide a sufficient condition when an exact reconstruction is doable. The algorithm also returns a metadata-computable, query-specific error certificate that decomposes approximation error into propagated representation error, coefficient mismatch, and uncovered message mass. We derive tighter corollaries for contractive GNNs, mean aggregation, GCN, and GAT, as well as a prediction-preservation certificate. The same certificate induces an error-weighted, monotone submodular objective for view selection. The algorithm scales the result from selected, minimized GMVs. To best reuse GMVs and reduce memory and inference overhead, we introduce a forgetting curve model to keep useful GMVs as long as possible.

(3) We experimentally verify the effectiveness and efficiency of the proposed GMV framework. Our results demonstrate that with *graph-model-as-a-view*, our methods can substantially reduce the inference cost of representative GNNs such as

GCNs, GAT, and GraphSAGE by 75.21%-90.53% on average across the studied large-scale graphs, with little to no loss (at most 1.18% and an average of 0.37% loss across all tested large-scale datasets) in inference accuracy. We further showcase the practicality of GMV through case studies in real-world scenarios, including cyberattack detection and anomaly detection in Bitcoin transactions.

Related Work. We categorize the most relevant work below.

View-based Graph Querying. Graph views have been studied for graph-pattern evaluation [12]–[15]. Fan et al. [12] introduced view-based evaluation for graph pattern queries; Li et al. [13] developed lower and upper approximations for subgraph search under approximate matching; Song et al. [14] applied views to knowledge-graph querying; and Zhang et al. [15] proposed automatic view selection using extended graph views that preserve edge-induced subgraphs for subgraph and supergraph queries.

These studies focus on evaluating graph queries rather than graph-model inference. In contrast, we introduce view structures that package memory graphs, cached model outputs, and reconstruction metadata, together with algorithms for selecting, minimizing, and assembling such views to answer inference queries. Existing graph-query view techniques therefore cannot be directly applied to our setting.

Accelerating Graph Inference. Existing approaches generally fall into two categories. (1) Model-centric methods optimize model execution. Pre-propagation methods such as SIGN [16], PPRGo [17], and SCARA [18] decouple neighborhood aggregation from inference by precomputing multi-hop features. GKD [19] distills a full-graph teacher GNN into a smaller student model, while Channel Pruning [20] removes redundant model channels. These methods modify the inference pipeline or rely on model-specific designs. (2) Data-centric methods simplify the input graph. NeuralSparse [21] and AdaptiveGCN [22] learn to prune task-irrelevant edges, whereas Dspar [23] uses resistance-based importance measures. More recently, SPGC [24] compresses graphs with inference-friendly operators. Unlike graph pruning or compression alone, our work exploits external or historical graph computations as queryable serving objects. A GMV couples a memory graph with model outputs and metadata that support exact or approximate reconstruction with bounded error.

Federated Graph Learning. Federated learning methods such as FGSSL [25] address structural heterogeneity and non-IID local data, often by transferring knowledge between global and local models. Recent surveys summarize distributed and parallel graph learning [26], [27], including data parallelism [28], [29], model parallelism [30], [31], and hybrid execution [32], [33]. These methods improve efficiency by distributing training or inference over partitions of the workload.

Our setting differs in two respects. First, we focus on serving a fixed graph model by reusing historical inference results, rather than training or generalizing local models. Second, GMV memory graphs need not be partitions or subgraphs of

the queried graph, although our algorithms can benefit from existing graph-parallel execution strategies.

Our experimental evaluation compares GMV-based inference against SPGC [24], and Dspar [23] as direct baselines, while Channel Pruning [20] and GKD [19] are included as case studies of representative model- and learning-based methods. We report additional details in the full version of the paper [34].

II. GRAPHS AND GRAPH MODELS

Graphs. A graph $G = (V, E)$ has a set of nodes V and a set of edges $E \subseteq V \times V$. Each node v carries a tuple $T(v)$ of attributes and their values. The size of G , denoted as $|G|$, refers to the total number of its edges, i.e., $|G| = |E|$. Given a node v in G , the L -hop neighbors of v , denoted as $N^L(v)$, refers to the set of all the nodes in the L -hop of v in G . The L -hop subgraph of a set of nodes $V_s \subseteq V$, denoted as $G^L(V_s)$, is the subgraph induced by the node set $\bigcup_{v_s \in V_s} N^L(v_s)$.

Graph Models. A graph (ML) model $\mathcal{M}$ is a function that takes as input a featurized representation $G = (X, A)$ to an output embedding matrix Z , i.e., $\mathcal{M}(G) = Z$. Here X is a matrix of node features, and A is a (normalized) adjacency matrix of G .² In this work, we consider graph neural networks (GNNs) and show that our methods are generally applicable to mainstream GNNs and their specifications.

Graph Inference. We follow a query language perspective [36], [37] to characterize the inference process of a graph model. Given a GNN $\mathcal{M}$ with L layers, each node v is associated with a layer-specific *node update operator* M_v^k at each layer k ($k \in [1, L]$), defined as:

$$M_v^k(\Theta^k, \text{AGG}(X_u^{k-1}, X_v^{k-1}, \forall u \in \mathcal{N}(v)))$$

M_v^k computes X_v^k , which refers to the output node representation at the k -th layer of $\mathcal{M}$ ($k \in [1, L]$), determined by (1) the learned model parameters Θ^k , (2) an aggregation function AGG (e.g., $\sum$, CONCAT), and (3) the previous layer embeddings of the neighbors of v that participate in the inference at the k -th layer (denoted as $\mathcal{N}(v)$) and v itself. When $k = 1$, $X_v^0 = X_v \in X$, i.e., the input features.

The *inference process* of a GNN $\mathcal{M}$ with L layers takes as input a graph $G = (X, A)$, and computes the embedding X_v^k for each node $v \in V$ at each layer $k \in [1, L]$, by recursively applying layer-specific M_v^k . Hence, the inference process can be considered as the computation of a composition of *node update operators*:

$$X_v^k = M_v^k \circ M_v^{k-1} \circ \dots \circ M_v^1(X_v)$$

A GNN $\mathcal{M}$ has a *fixed* inference process, if its node update function is specified by fixed input model parameters Θ , layer numbers L , and aggregator. It has a *deterministic* inference process, if it generates the same output for the same input.

²A feature vector X_v of a node v can be a word embedding or one-hot encoding [35] of $T(v)$. A is often normalized as $\hat{A} = A + I$, where I is the identity matrix.

We consider GNNs with fixed, deterministic inference processes. Such GNNs are desired for generating consistent and robust outputs. For simplicity, we assume that M_v specifies a proper set of neighbors that participate in the inference process as $N(v) \subseteq \{u | (u, v) \in E \text{ or } (v, u) \in E\}$. This formulation allows us to cope with GNNs that exploit neighborhood sampling (such as GraphSAGE), and directed or asymmetric message passing, where the effective neighborhood is determined by the inference rule rather than the raw graph structure. For most representative spatial and message-passing GNNs, the inference process consists of layered aggregation and transformation operations and can be computed in PTIME [38].

Bounded computation scope. For a target node v_t , an L -layer message-passing GNN depends only on the participating nodes and edges in $G^L(v_t)$. This bounded computation neighborhood captures the scope against the view structure in our view-based framework (see Section III-A). We do not assume access to global spectral decompositions or other operators whose output depends on the complete graph.

GNN Classes. A set of fixed, deterministic GNNs $\mathbb{M}$ belongs to a *class* of GNNs $\mathbb{M}^L$, if for every GNN $\mathcal{M} \in \mathbb{M}$, (1) $\mathcal{M}$ has L layers, and (2) $\mathcal{M}$ uses the same form of node update function M_v^k , for each node $v \in V$ and $k \in [1, L]$. Table I summarizes mainstream GNNs with node update functions. Other GNNs are mostly their variants that differ in the choices of aggregation function and other PTIME computable operators.

Inference Query. An *inference query* Q is a triple $Q = (M, v_t, G)$ where G is the input test graph, M is a fixed, deterministic GNN model from a class of GNNs $\mathcal{M}^L$ with L layers, and v_t (can be $\emptyset$) is a designated target node. In practice, to answer Q , the model M may provide an API to be invoked, which requires as input $G = (X, A)$ and perform an inference process to return a task-specific result based on the output-layer embedding $Z(v_t) = \mathbf{x}_{v_t}^{(L)}$.

We define the *output* of Q as (a) the embedding matrix Z , if v_t is not specified *i.e.*, $Q=(M, \emptyset, G)$; or (b) $Z(N^L(v_t)) = \bigcup_{v \in N^L(v_t)} Z(v)$, *i.e.*, a set of embeddings that includes all participating nodes in the inference computation of v_t , which involves up to $N^L(v_t)$ in G , for any GNN $M \in \mathcal{M}^L$ with L layers. The result Z is often referred to as the *logits vectors*, and can be further processed (e.g., via softmax) to produce prediction scores or labels. As the inference process is fixed and deterministic, the output of Q is determined by G and M .

III. VIEW-BASED GRAPH INFERENCE

To support view-based inference, we introduce a view structure, notably, *graph model views* (GMVs). Given an inference query $Q(M, v_t, G)$, GMVs are a set of shared *memoization* structures that record output and necessary metadata of an inference of M over a set of published memory graphs, to be utilized to reconstruct $Q(M, v_t, G)$.

A. Graph Model Views: A Characterization

Given a L -layered GNN M , a *graph model view* (GMV) is a triple $\mathcal{V} = (G_{\mathcal{V}}, \mathcal{T}_{\mathcal{V}}, Z_{\mathcal{V}})$, which contains (a) a *memory graph*

$G_{\mathcal{V}}$, a graph over which an inference of M is performed, (b) a memoization structure $\mathcal{T}_{\mathcal{V}}$, to store the auxiliary metadata (to be discussed); and (c) $Z_{\mathcal{V}}$, a layer-aware cache $\{X_{\mathcal{V}}^0, \dots, X_{\mathcal{V}}^L\}$ (or equivalent sufficient aggregate messages), for fast message reconstruction (see Section III-C); $X_{\mathcal{V}}^L$ is the final output representation of M over $G_{\mathcal{V}}$.

In an analogy of database views, we say a GMV $\mathcal{V}$ is a *virtual view* if it only contains a memory graph, *i.e.*, $\mathcal{V} = (G_{\mathcal{V}}, \emptyset, \emptyset)$, while the structure $\mathcal{T}_i$ and output Z_i are to be derived as needed; Otherwise, it is *extended*, with pre-computed $\mathcal{T}_i$ and Z_i ready to be served.

Remarks. GMVs are consistently characterized by the semantics of inference queries Q . Two different GMVs may (1) share a same memory graph with different test nodes, (2) involve the same test nodes with different memory graphs, or (3) agree on the test nodes and memory graph, yet bookkeeping different outputs from different GNNs of a class $\mathcal{M}^L$.

View-based Graph Inference. The paradigm of *view-based graph inference* has a general form:

- **Input:** A graph G , a GNN $M \in \mathcal{M}$, an inference query $Q(M, v_t, G)$, and a set of GMVs $\mathcal{V}$.
- **Output:** an approximated result of Q by referring to $\mathcal{V}$, rather than a direct inference over G from scratch.

Specifically, (1) if $\mathcal{V}$ are virtual GMVs, one invokes at most $|\mathcal{V}|$ local inference queries over the memory graphs $\mathcal{G}_{\mathcal{V}} = \bigcup G_{\mathcal{V}_i}$; or (2) Otherwise, assemble the extended GMVs (with Z_i , $\mathcal{G}_{\mathcal{V}_i}$ and $\mathcal{T}_{\mathcal{V}_i}$ from each GMV $\mathcal{G}_{\mathcal{V}_i}$ for reconstruction.

B. System Model and Serving Workflow

Our primary deployment setting separates a graph owner from a GMV serving layer with an offline phase and an online serving workflow as follows.

Offline phase. In an offline phase, the owner or authorized provider runs M on memory graphs, materializes $Z_{\mathcal{V}}$, constructs $\mathcal{T}_{\mathcal{V}}$, and optionally minimizes and indexes the resulting views (see Section V for view minimization);

Online serving workflow. For each request, (1) the graph owner locally inspects the bounded computation neighborhood $G^L(v_t)$; (2) an owner-side or trusted matching procedure compares $G^L(v_t)$ with candidate memory graphs $G_{\mathcal{V}}$, computes the match relation R^L , and derives the model-specific message-reconstruction metadata recorded in $\mathcal{T}_{\mathcal{V}}$, such as match multiplicities, scaling coefficients, degree-normalization terms, and uncovered-message statistics; (3) the serving layer selects a set of GMVs (Section IV); (4) the selected memory graphs $G_{\mathcal{V}}$, auxiliary metadata $\mathcal{T}_{\mathcal{V}}$, and cached representations $Z_{\mathcal{V}}$ are used to assemble the required layer inputs and apply the original model update functions; and (5) the system falls back to partial or direct inference when view coverage or the reconstruction-confidence certificate is insufficient. Steps (1)–(2) require “once-for-all” target-local access, whereas Steps (3)–(4) operate only on the derived matching metadata and the selected GMVs.

The online phase avoids a complete inference pass over G , but access $G^L(v_t)$ locally to compute matching metadata. Its latency is therefore determined by local matching, reconstruction over selected memory graphs, and cache maintenance. The method is post-hoc and requires no retraining.

Remarks. The framework reduces repeated inference and raw-graph transfer, but it does not by itself provide a formal privacy guarantee. Match sets, degrees, embeddings, or correction coefficients may reveal information and should be protected by deployment-specific access control or privacy mechanisms. External public graphs and federated repositories are compatible secondary sources of memory graphs, provided that the owner can compute the required local metadata.

C. View-based Inference: Doability

We next show that the view-based graph inference is doable, by providing a *sufficient* condition that is checkable in PTIME.

General Idea. Given a test node $v_t \in G$, our idea considers graph inference computation for v_t over G as the “observer”, and identify the node pairs between G and G_V that cannot be distinguished by the inference process, such that it can be computationally simulated by the inference over G_V directly, leveraging properly stored neighborhood statistics.

We start by introducing an *embedding-respecting L -bisimulation* relation to capture inference-equivalence within L -hop neighbors of a designated node. Bisimulation [39] is a fundamental notion that establishes when two nodes (states) in a transition graph cannot be distinguished by an external observer, and has been exploited in data compression for graph computation [24]. Given two graphs G_V and G , a test node v_t in G , and an integer L , let $G^L(v_t)$ be the L -hop subgraph of v_t in G . An *L -bisimulation at v_t* , denoted as $R_{v_t}^L$, is a binary relation between the nodes of G_V and $G^L(v_t)$, such that

- (1) there exists a node v'_t in G_V where $X_{v_t}^0 = X_{v'_t}^0$; and
- (2) for each pair $(v, v') \in R_{v_t}^L$, (a) $X_v^0 = X_{v'}^0$, and (b) for every edge (u, v) in $G^L(v_t)$, there is a match (u', v') in $G_V^L(v'_t)$, such that $(u, u') \in R_{v_t}^L$; and for every edge (u', v') in $G_V^L(v'_t)$, there is a match (u, v) in $G^L(v_t)$, such that $(u, u') \in R_{v_t}^L$.

We say v'_t and v_t are *L -bisimilar* if an L -bisimulation between $N^L(v'_t)$ and $N^L(v_t)$ contains (v_t, v'_t) . When v_t is specified, we denote $R_{v_t}^L$ as R^L for simplicity. For each node v (resp. v') in $G^L(v_t)$ (resp. $G_V^L(v'_t)$), we define the *match set* of v (resp. v'), denoted as $R^L(v)$ (resp. $R^L(v')$), as $\{v' | (v, v') \in R^L\}$ (resp. $\{v | (v, v') \in R^L\}$). The *match set* of each edge $e = (u, v)$ in $G^L(v_t)$ (resp. $e' = (u', v')$ in $G_V^L(v'_t)$), denoted as $R^L(e)$ (resp. $R^L(e')$), are defined similarly.

We can verify the following properties of L -simulation.

Lemma 1: *Given G with target v_t , a memory graph G_V , and integer L , there exists a unique maximum embedding-respecting L -bisimulation relation R^L between $G^L(v_t)$ and the matching neighborhoods in G_V , which can be computed in $O(|G_V| + |N^L(v'_t)| |G^L(v_t)|)$ time.* $\square$

Algorithm 1 : Layer-wise procedure GVInf

Input: Query $Q(M, v_t, G)$; a set of extended GMVs $\mathcal{V}'$; owner-supplied correspondence metadata $\mathcal{T}_Q$;
Output: an approximate output of $Q(M, v_t, G)$;
1: Retrieve layer-0 representatives and sufficient message statistics from $Z_{\mathcal{V}'}$ and $\mathcal{T}_Q$ from $\mathcal{V}'$;
2: **for** $k = 1, \dots, L$ **do**
3: **for** each target equivalence class c induced by L -bisimulation R^L participating at layer k **do**
4: assemble $\hat{m}_c^k$ from all matched nodes using multiplicities and model-specific coefficients in $\mathcal{T}_Q$;
5: compute $\hat{X}_c^k := M_c^k(\hat{m}_c^k, \hat{X}_c^{k-1})$ with the original node-update function;
6: **return** $\hat{Z}(v_t) := \hat{X}_{v_t}^L$ and requested post-processing;

Fig. 2: Layer-wise inference from selected views. Embeddings are reconstructed layer by layer. Extension phase not shown.

One can verify that L -bisimulation is reflexive, symmetric, and transitive. The maximum L -simulation can be computed by a verification procedure (denoted as L-Bisim, not shown) that (1) verifies the existence of v'_t in $O(|G_V|)$ time, and (2) computes the maximum bisimulation relation between $N^L(v'_t)$ and $N^L(v_t)$. This can be implemented by invoking fast bisimulation algorithms [40], [41], in $O(|N^L(v'_t)| |G^L(v_t)|)$ time. Here L is in practice small for GNNs.

Theorem 2: (Sufficient condition for view-based inference). *Given a fixed deterministic L -layer GNN M , an inference query $Q(M, v_t, G)$ can be approximately answered by only referring to an extended GMV $\mathcal{V}$, if there exists a matched node $v'_t \in G_V$ $((v_t, v'_t) \in R^L)$ such that, for every participating node v and layer of inference at v_t in G , $\mathcal{T}_V$ maps their incoming messages to a counterpart of an inference of M for v'_t over G_V , at a participating, inference-equivalent node in the match set of v in $G^L(v'_t)$, with associated multiplicities and model-specific normalization coefficients.* $\square$

We next provide a constructive proof of Theorem 2. Assume the message-preservation condition of Theorem 2 holds; We clarify (a) the construction of $\mathcal{T}_V$ for GMV $\mathcal{V}$, if $\mathcal{V}$ is virtual; and (b) provides a procedure, denoted as GVInf, to compute $Q(M, v_t, G)$ by referring to the extended GMV $\mathcal{V}$ only. This procedure is used as a “building block” for our general view-based inference algorithms, when the sufficient condition may not hold for a single GMV (see Section IV).

View-based Graph Inference. We outline the procedure GVInf. It takes as input an inference query $Q(M, v_t, G)$, $N^L(v'_t)$, and the maximum L -bisimulation R^L computed by procedure L-Bisim, and generates an output $Z'(v_t)$ as an approximate output of Q . It performs two steps below.

(1) **Extension.** If $\mathcal{V}$ is virtual, GVInf performs a “once-for-all” computation to extend $\mathcal{V}$ by constructing $\mathcal{T}_V$ and Z_V . (a) Given the node update function M_v , $\mathcal{T}_V$ is an abstract representation of R^L and relevant metadata, which contains:

GNNs	Node update function M_v on G	Rewriting with $M_{v'}$	Notes
Vanilla [4]	$X_v^k = \sigma(\Theta \cdot \text{AGG}(X_u^{k-1}, u \in \mathcal{N}(v)))$	$X_v^k \approx \sigma(\Theta \cdot \text{AGG}(\textcolor{red}{S_{e'}(v', u')} X_{u'}^{k-1}, u' \in \mathcal{N}(v')))$	$\text{AGG} \in \{\sum, \text{AVG}\}$; for AVG , apply RF_v
GCN [42]	$X_v^k = \sigma\left(\Theta^k \sum_{u \in \mathcal{N}(v)} \frac{X_u^{k-1}}{\sqrt{\deg_u \deg_v}}\right)$	$X_v^k \approx \sigma\left(\Theta^k \sum_{u' \in \mathcal{N}(v')} \frac{\textcolor{red}{S_{e'}(v', u')}}{\sqrt{\deg_{v'}}} X_{u'}^{k-1}\right)$	$\deg_{v'}$ is the degree of v' in G_V ; topology-sensitive
GAT [43]	$X_v^k = \sigma\left(\sum_{u \in \mathcal{N}(v)} \alpha_{uv} \Theta^k X_u^{k-1}\right)$	$X_v^k \approx \sigma\left(\sum_{u' \in \mathcal{N}(v')} \textcolor{red}{S_{e'}(v', u')} \Theta^k X_{u'}^{k-1}\right)$	Weight-sensitive; exact attention may require target-local recomputation
GraphSAGE [44]	$X_v^k = \sigma(\Theta^k \cdot (X_v^{k-1} \parallel \text{AGG}(X_u^{k-1}, u \in \mathcal{N}(v))))$	$X_v^k \approx \sigma(\Theta^k \cdot (X_{v'}^{k-1} \parallel \text{AGG}(\textcolor{red}{S_{e'}(v', u')} X_{u'}^{k-1}, u' \in \mathcal{N}(v'))))$	$\parallel$: concatenation; $\text{AGG} = \text{AVG}$
GIN [45]	$X_v^k = \sigma\left(\text{MLP}\left((1 + \gamma) X_v^{k-1} + \sum_{u \in \mathcal{N}(v)} X_u^{k-1}\right)\right)$	$X_v^k \approx \sigma\left(\text{MLP}\left((1 + \gamma) X_{v'}^{k-1} + \sum_{u' \in \mathcal{N}(v')} \textcolor{red}{S_{e'}(v', u')} X_{u'}^{k-1}\right)\right)$	Sum aggregation with multiplicity correction

TABLE I: Model-specific message rewriting for representative GNN classes. Scaling factors are highlighted in **red**, where $v \in R^L(v')$.

- a map that compactly encodes R^L , where for each node v' in the memory graph G_V , it records its match set $R^L(v')$;
- a map $(e', S_{e'})$, where for each edge $e' = (v'_1, v'_2)$ in $N^L(v'_t)$, $S_{e'}$ is a *normalized match size*, defined as $\frac{|R^L(e')|}{|R^L(v'_2)|}$, where v'_2 w.l.o.g. is closer to v'_t .

It then performs a local inference of M over $G^L(v'_t)$ to store Z_V . This completes the construction of an extended GMV.

(2) **Reconstruction.** This step reconstructs $Z'(v_t)$ with *only* G_V , $\mathcal{T}_V$ and Z_V . It has two steps below.

(a) *Message rewriting.* For each node v' in $N^L(v'_t)$, GVInf looks up the set $R^L(v') = \{v | (v, v') \in R^L\}$ in $\mathcal{T}$, the extended output $Z(v')$, and $S_{e'}$ for each edge (v', u') in G_V . For each node v in $R^L(v')$, its embedding X_v^k at layer k ($k \in [1, L]$) is updated as

$$M_{v'}^k(s(e'), X_{u'}^{k-1}), \forall e' = (u', v')$$

where $s(\cdot)$ is a scaling function that ad-hocly derives a *scaling factor*, determined by the specific aggregator of M_v' (e.g., $\sum$, AVG) that aggregates neighborhood metadata of v' (such as node degrees, edge attentions, etc.), the normalized edge match size $S_{e'}$, and model weights Θ (which is known and remain unchanged in the given update function M^k). These metadata are all bookkept in $\mathcal{T}_V$ from $\mathcal{V}$. We summarize the scaling function $s(\cdot)$ (marked in red) in Table I for mainstream message-passing GNNs³.

(b) *Approximating output.* GVInf next performs layer-wise reconstruction. For the test node v_t , the inference computes $Z'(v_t)$ by iteratively apply the same node update function, yet with rewritten messages in (a) over $G^L(v'_t)$, from $k = 1$ to L .

Remarks. Table I illustrates a model class-specific reconstruction template. For sum/mean aggregation, counts and multiplicities are sufficient; for GCN, the relevant target normalization coefficients should also be preserved; for neighborhood-sampled GraphSAGE, the sampled multiset should be represented. The GAT row is approximate when S_e substitutes for

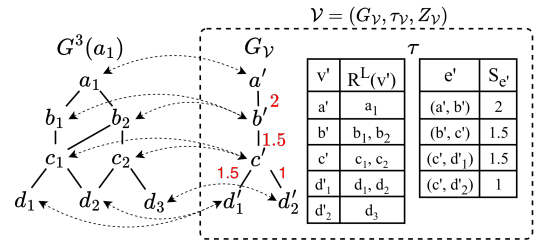


Fig. 3: A illustration of View-based Inference. The inference for test node a_1 in $G^3(a_1)$ is “simulated” by a counterpart for a' that 3-bisimilar to a_1 in a GMV, for GNN class $\mathcal{M}^3$.

targeted attention. It requires recomputed attention or stored sufficient inputs to reproduce its softmax normalization.

Example 3: Consider the 3-layer Vanilla GNN query $Q(M, a_1, G)$ and the memory graph in Fig. 3. Procedure GVInf first computes the relation $R^3 = \{(a', a_1), (b', b_1), (b', b_2), (c', c_1), (c', c_2), (d'_1, d_4), (d'_1, d_2), (d'_2, d_3)\}$, and materializes the corresponding layer-wise representations and edge scaling factors. During reconstruction, for example, the three target edges represented by (b', c') and the two matched copies of b' yield $S_{e'}(b', c') = 3/2$. The rewritten update uses this factor before applying the original nonlinear node-update function. The remaining layers are reconstructed analogously. For GAT, a stored scalar is only approximate unless the target-context attention normalization can be reproduced. $\square$

Correctness and Error Bounds. Under the condition of Theorem 2, correctness follows by induction on layers. At layer 0, matched nodes have identical features. If their representations agree through layer $k - 1$, the metadata in $\mathcal{T}_V$ reconstructs the same incoming representations and model-specific coefficients at layer k ; the deterministic update therefore produces the same result. Repeating through layer L yields the exact output at v_t . Otherwise, the procedure returns an approximation.

Certified error bound. Consider the practical assumption that the layer update is Lipschitz, the reconstruction error is then controlled by the difference between the target and recon-

³Here v may have multiple matches from GMVs. Algorithm GVInf breaks the tie by choosing an arbitrary $v' \in R^L(v)$.

structed messages. Let C_v^k and U_v^k be the “covered” (have match set in R^L) and uncovered (have no match in R^L) neighbors of v at layer k , and let $\hat{a}_{vu}^k$ denote the coefficient reconstructed from $\mathcal{T}_V$ for a message from a covered node u .

Lemma 3: Assume the update of v is $K_{v,k}$ -Lipschitz and $\|X_u^{k-1}\| \leq H_u^{k-1}$. The node-wise error satisfies

$$e_v^k \leq K_{v,k} \left(\sum_{u \in C_v^k} |\hat{a}_{vu}^k| e_u^{k-1} + q_v^k + r_v^k \right),$$

where q_v^k bounds coefficient mismatch on covered messages and r_v^k bounds the contribution of messages of uncovered nodes. These quantities are stored in, or upper-bounded from, $\mathcal{T}_V$; recursively applying the inequality gives a query-specific output-error certificate. $\square$

Under Theorem 2, the initial, coefficient, and uncovered-message errors are all zero, so exactness follows by induction. Tighter model-specific forms for mean/sum aggregation, GCN, GAT, and contractive GNNs follows from Theorem 2 and Lemma 3. Due to limited space, we present the detailed proofs, and model-class specific results in the Appendix of [34].

Time Cost. We separate one-time offline cost and online costs. Extending virtual views incurs model inference, metadata construction, and optional minimization over the memory graphs. For an online query, matching costs $T_{\text{match}}(G^L(v_t), \mathcal{V})$, greedy selection costs $O(k|\mathcal{V}|)$ after coverage sets have been computed, and reconstruction costs $O(L|G_{\mathcal{V}'}|)$ for selected views $\mathcal{V}'$. Thus the online cost is

$$T_{\text{match}} + O(k|\mathcal{V}|) + O(L|G_{\mathcal{V}'}|),$$

plus cache maintenance. It is independent of the full $|G|$ only when the required target-local neighborhood processing and matching are bounded by $G^L(v_t)$ and do not scan the complete graph. The reported experiments include the preprocessing convention stated in Section VII.

The above results can be readily extended to a set of GMVs $\mathcal{V} = \{\mathcal{V}_1, \dots, \mathcal{V}_n\}$. Define the memory graph $\mathcal{G}_{\mathcal{V}}$ of $\mathcal{V}$ as the union of the memory graphs $G_{\mathcal{V}_i}$ of $\mathcal{V}_i \in \mathcal{V}$.

Corollary 4: An inference query $Q(M, v_t, G)$ can be answered by only referring to a set of GMVs $\mathcal{V}$, if there exists a node v'_t in $\mathcal{G}_{\mathcal{V}}$ that is L -bisimilar with v_t in $G_{\mathcal{V}}$. $\square$

Example 4: The question in Example 1 can be represented by an inference query $Q(M, v_a, G)$. A set of GMVs $\mathcal{V}$ can be provided, including two virtual GMVs $\mathcal{V}_1$ and $\mathcal{V}_2$, with memory graphs induced from two separately posted sample graphs about Cyclobutadiene (C_4H_4) and methylhydroxylamine (CH_5NO), respectively. The extended GMV $\mathcal{V}_1$ sets $\mathcal{T}_{\mathcal{V}_1}$ as two key-value lists that store L -bisimilar node pairs, where keys are nodes in G_1 , and values are nodes in the original G . For example, (v_2, C_2) is a L -bisimilar pair. Z_1 is the precomputed result $M(G_1, v_1)$ that represent inference result of node v_1 over G_1 . Similarly for the extended GMV

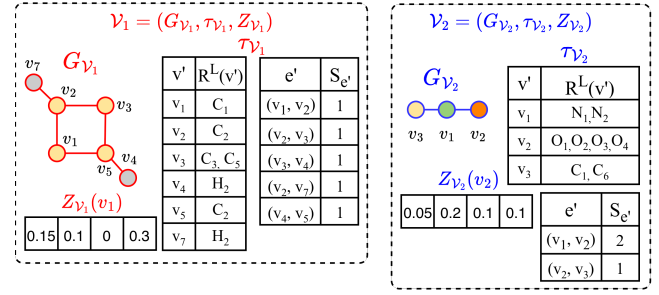


Fig. 4: Extended GMVs $\mathcal{V}_1$ and $\mathcal{V}_2$.

Algorithm 2 : GVSel

Input: Graph G , inference query $Q(M, v_t, G)$, a set of views $\mathcal{V}$, a test node v_t , maximum number of views k ;
Output: Set of views $\mathcal{V}'$ that best covers $Q(G, M, v)$;
1: set $\mathcal{V}' := \emptyset$, set $U := \emptyset$;
2: $\forall \mathcal{V}_i \in \mathcal{V}$, compute $C[\mathcal{V}_i]$, /*set of nodes in $N^L(v)$ covered*/;
3: **while** $\exists \mathcal{V}_i \in \mathcal{V} \setminus \mathcal{V}' : |S[\mathcal{V}_i] \setminus U| > 0$ or $|\mathcal{V}'| < k$ **do**
4: $\forall \mathcal{V}_i \in \mathcal{V} \setminus \mathcal{V}'$, $S[\mathcal{V}_i] := S[\mathcal{V}_i] \setminus U$;
5: $\mathcal{V}^* := \arg \max_{\mathcal{V}_i \in \mathcal{V} \setminus \mathcal{V}'} |S[\mathcal{V}_i]|$ where $S[\mathcal{V}_i] \neq \emptyset$;
6: $\mathcal{V}' := \mathcal{V}' \cup \{\mathcal{V}^*\}$;
7: $U := U \cup S[\mathcal{V}^*]$;
8: **return** $\mathcal{V}'$;

Fig. 5: Algorithm GVSel.

$\mathcal{V}_2$. As there exists a 2-bisimilar relation between the union of the two memory graphs $(\mathcal{G}_{\mathcal{V}})$ and $G^2(a_1)$, one can use $\mathcal{V}$ with scaling factors to approximate $Q(M, a_1, G)$ in Example 1. $\square$

Remarks. In practice, a GMV can be constructed with subgraphs from published graph data as needed. For example, the memory graphs $G_{\mathcal{V}_1}$ and $G_{\mathcal{V}_2}$ in Fig. 4 are useful subgraphs of G_1 and G_2 . This allows a better exploitation of external, open graph data having reusable fractions as long as they include an L -bisimilar nodes.

We next investigate three fundamental problems: view selection, view minimization, and a general algorithm for inference using views when the coverage condition may not hold. This gives a comprehensive treatment of our proposed paradigm.

IV. VIEW SELECTION

For an inference query, one may obtain multiple sample graphs as (virtual) GMVs $\mathcal{V}$ from organizations or data platforms such as Hugging Face [7] and OpenML [46]. To ensure fast inference and reduce e.g., the number of function calls of M inference, we investigate view selection problem.

View Selection. Given an inference query $Q(M, v_t, G)$, and a set of GMVs $\mathcal{V}$, we want to find a smallest set $\mathcal{V}' \subseteq \mathcal{V}$, such that $\mathcal{G}_{\mathcal{V}'}$ satisfy the sufficient condition (Lemma 4).

We show that this problem is NP-hard even for a two-layer sum-aggregation GNN with discrete node features. A reduction can be constructed from the set cover problem

(see [34]). We next provide an algorithm, denoted as GVSel, for view selection problem.

Selection Strategy. Our algorithm restates view selection by testing a “coverage” property of each GMVs. Given the memory graph $G_{\mathcal{V}}$ of a GMV $\mathcal{V}_i \in \mathcal{V}$, it first computes a non-empty L -bisimilar relation R^L between $G_{\mathcal{V}}^L(v_t)$ and $G^L(v_t)$, for $L \in [0, L]$ (by invoking procedure L-Bisim), and with l the largest possible value (up to L). Here a 0-bisimilar relation R^0 exists, if there is a node v'_t in $G_{\mathcal{V}}$ that has the same feature vector as v_t . For each $\mathcal{V}_i$ that contains a node v'_t that is L -bisimilar with v_t , it sets a “cover” set $S[\mathcal{V}_i]$ as the match set of nodes and edges induced by the L -bisimilar relation R^L .

The algorithm GVSel computes a k -set of GMVs $\mathcal{V}' \subseteq \mathcal{V}$ such that either (1) the union of their memory graphs $G_{\mathcal{V}}$ satisfy the condition in Lemma 4, or otherwise, (2) it chooses a set of GMVs, such that $G_{\mathcal{V}}$ covers $G^L(v_t)$ via maximal l -bisimilar relation, hence missing small amount of edges involved in the inference of $Q(M, v_t, G)$, to approximately answer Q . We illustrate the pseudo-code of GVSel in Fig. 5, and present detailed description in [34].

Example 5: In Fig. 6, $\mathcal{V}_1$ alone covers the complete two-hop target neighborhood. Under budget $k = 2$ and candidates $\{V_2, V_3, V_4\}$, GVSel first chooses V_2 , which has the largest coverage, and then V_4 , which contributes the largest uncovered portion. It returns $\{V_2, V_4\}$, as V_3 adds no more coverage. $\square$

Analysis. The coverage objective is monotone and submodular; hence the greedy algorithm achieves the standard $(1 - 1/e)$ approximation for selecting at most k views with maximum weighted coverage of $G^L(v_t)$. Once coverage sets are materialized, the greedy phase incurs $O(k|\mathcal{V}|)$ cost; matching candidate views to $G^L(v_t)$ is accounted for separately. We present a detailed proof in [34].

V. VIEW MINIMIZATION

While view selection allow us to exploit a small set of reusable GMVs that maximize the coverage of $G^L(v_t)$ with their indistinguishable counterparts, treating inference process as an observer, nodes among the memory graphs themselves may contribute to redundancy and a source of inference and storage overhead, especially for large memory graphs.

We next introduce two preprocessing strategies to reduce the size of GMVs. The first compresses memory graphs to smaller representation. The second prunes those that are dominated by other GMVs in terms of coverage ability.

View compression. Our first strategy is to compress $G_{\mathcal{V}}$ with a stronger node equivalence relation that preserves the L -bisimilarity. To this end, We extend inference-friendly compression [24] Given a graph $G_{\mathcal{V}}$, we compute a node equivalence relation R^I between the nodes of $G_{\mathcal{V}}$ and nodes in G , such that for each node pair $(v, v') \in R^I$, (1) v and v' has the same input feature vector, and (2) for any neighbor u of v , there exists a neighbor u' of v' , such that $(u, u') \in R^I$. A compressed graph of $G_{\mathcal{V}}$ is the quotient graph $G_{\mathcal{V}}^c$ induced

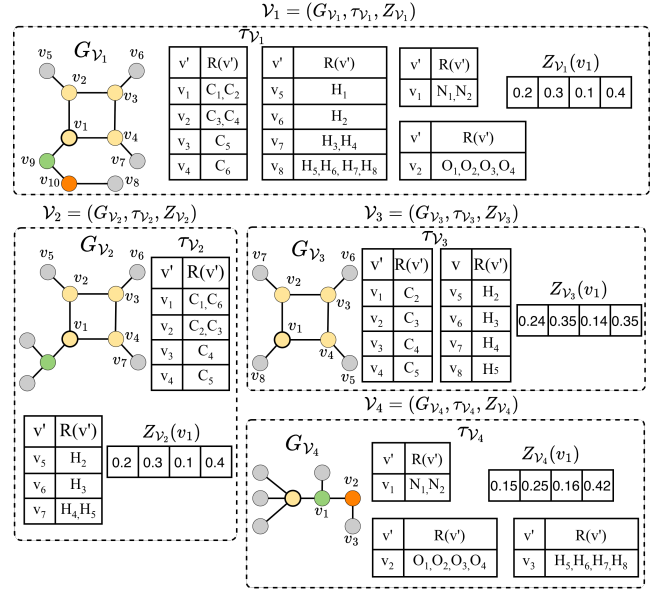


Fig. 6: View Selection for Graph Inference. Of the four GMVs $\mathcal{V}_1, \mathcal{V}_2, \mathcal{V}_3$, and $\mathcal{V}_4$, $\mathcal{V}_1$ completely covers Q while $\mathcal{V}_2, \mathcal{V}_3$, and $\mathcal{V}_4$ covers Q partially.

by R_I , which contains a set of nodes, where each node $[v]$ is an equivalent class of a node in $G_{\mathcal{V}}$; and there exists an edge $([v], [v'])$ if (v, v') is an edge in $G_{\mathcal{V}}$. Let $\mathcal{V}_c$ with $G_{\mathcal{V}}^c$ be a compressed GMV of $\mathcal{V}$, We have the following result.

These view minimization strategies effectively finds redundancy in view-based graph inference. Such redundancy can be mitigated by a “once-for-all” minimization of $\mathcal{V}$ to a counterpart that covers $G^L(v_t)$, hence in turn reduce inference cost as much as possible.

View Dominance. Our second strategy pre-prune GMVs that cannot cover more than another GMVs for inference query $Q(M, v_t, G)$. We extend graph simulation [47] for graph inference, denoted as $R_{\mathcal{V}}^I$. Given a graph $G_{\mathcal{V}}$ with node set V , $R_{\mathcal{V}}^I \subseteq V \times V$ is a binary relation such that for any pair $(v, v') \in R_{\mathcal{V}}^I$, (1) $X_v^0 = X_{v'}^0$, i.e., they have the same feature vector, and (2) for any neighbor u of v in $G_{\mathcal{V}}$, there exists a neighbor u' of v' in $G_{\mathcal{V}}$, such that $(u, u') \in R_{\mathcal{V}}^I$. In this case, we say v is *inference dominated* by v' , denoted as $v \prec v'$.

One can verify that $R_{\mathcal{V}}^I$ is a preorder (reflexive, transitive), and the maximal relation $R_{\mathcal{V}}^I$ can be efficiently computed by a slight revision of the algorithms that computes graph simulation [47] in quadratic time. We observe the following.

Lemma 5: For any inference query $Q(M, v_t, G)$ that can be answered by referring to $\mathcal{V}$ with $G_{\mathcal{V}}$, Q can be answered by $G_{\mathcal{V}} \setminus \{v\}$ if v is dominated by another in $G_{\mathcal{V}}$. $\square$

Algorithm. Given a set of (virtual) GMVs $\mathcal{V}$, the view minimization algorithm, denoted as GVMin, preprocesses $\mathcal{V}$ to a minimal representation $\mathcal{V}_m$ with a compressed memory graph $G_{\mathcal{V}}^m$ in two phases. GVMin has two phases. The canonicalization phase repeatedly refines a feature-respecting

Algorithm 3 : GVMin**Input:** A set of GMVs $\mathcal{V}$, inference query $Q(M, v_t, G)$;**Output:** A minimized GMV $\mathcal{V}_m$;

- 1: set $\mathcal{V}_m := (\mathcal{G}_m, \emptyset, \emptyset)$;
- 2: $\mathcal{G}_m := \bigcup G_i$ (G_i ranges over memory graphs in $\mathcal{V}$);
- 3: $\mathcal{P} = \{V_m\}$;
- 4: Refine $\mathcal{P} = \bigcup_{B \in \mathcal{P}} \text{Refine}(B, \mathcal{P})$ until convergence;
- 5: $\mathcal{G}_m := \mathcal{G}_m / \mathcal{P}$, /*collapse each block into equivalence classes*/;
- 6: set relation $R_{\mathcal{L}}^I := \emptyset$; compute $R_{\mathcal{L}}^I$ over $\mathcal{G}_m$;
- 7: Prune $\mathcal{G}_m$ by removing dominated nodes;
- 8: Update $\mathcal{V}_m$ with $\mathcal{G}_m$; update $\mathcal{T}_m$ and $\mathcal{Z}_m$ accordingly if not $\emptyset$;
- 9: **return** $\mathcal{V}_m$;

Fig. 7: Algorithm GVMin.

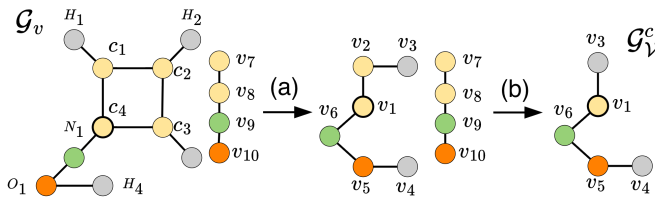


Fig. 8: View compression and pruning.

forward-bisimulation partition until a fixpoint and collapses each final block into one equivalence class [41], [48], [49]. The pruning phase then computes the inference dominance $R_{\mathcal{L}}^I$ and removes nodes whose represented message statistics are subsumed by those of another node. Details of the refinement procedure are given in [34].

Example 6: In Fig. 8, partition refinement merges the structurally and feature-equivalent nodes in the left component into six classes, while the four-node chain remains distinct. Dominance pruning then removes v_7, v_8, v_9, v_{10} , because their represented neighborhoods are subsumed by v_2, v_1, v_6, v_5 , respectively. This yields a memory graph with five nodes and four edges, preserving the represented inference coverage. $\square$

Analysis. By Lemma 5, model-aware equivalence and dominance pruning preserve the message statistics from the views. If the selected memory graph has $|V_m|$ nodes and $|E_m|$ edges, partition refinement and dominance testing take $O(|E_m||V_m|)$ time, in addition to constructing the union of views.

VI. APPROXIMATE INFERENCE QUERY WORKLOAD

We next provide a general algorithm, denoted as GMVInf, to approximately evaluate a set of inference queries. This extends our discussion to cope with a set of test nodes V_T .

Algorithm outline. GMVInf processes a stream of queries $Q(M, v_t, G)$. Offline, it minimizes and materializes candidate GMVs. For each query, it invokes GVSel to choose at most k views, applies GVInf to reconstruct the requested output, and returns a partial/direct-inference fallback when coverage is insufficient. The full pseudocode is provided in the appendix.

To maximize the reusability of GMVs and further reduce the selection cost, algorithm GMVInf dynamically maintains a cache of memory graph following a forgetting principle [50], [51]. The cache periodically swaps out memory graphs that are no longer “fresh” and retain those expected to be needed, determined by GVSel and a forgetting curve expressed by an exponential decay model. We provide details and an ablation study of its impact to the performance of view-based inference (see the accompanying technical report [34]).

Analysis. For each query, let T_{match} be the cost of constructing match and coverage metadata. Given these metadata, selection costs $O(k|\mathcal{V}|)$, reconstruction costs $O(L|G_{\mathcal{V}_m}|)$, and cache maintenance costs T_{cache} . One-time GMV materialization and minimization are amortized across the workload.

VII. EXPERIMENTAL STUDY

Using real-world benchmark datasets, we evaluated GMVInf and its components (GVSel, GVMin, GVInf). Our source code and datasets are made available⁴.

A. Experimental Settings

Datasets. We employ seven real-world graph benchmark datasets (summarized in Table. II). (1) **Cora** [52], a citation network with node features as 0/1-valued word vector indicating the absence/presence of keywords from a dictionary; (2) **Elliptic++** [53] is a Bitcoin transaction dataset, where nodes correspond to wallet addresses and edges represent cryptocurrency transfers, for classifying illicit transactions; (3) **ATLAS** [54], is a cybersecurity log dataset for attack investigation. Each node represents an event (*e.g.*, system or network activity), and edge denote temporal or causal relationship between events; (4) **Protein** [55], is a molecular graph dataset with 1,113 graphs. Node features include a 3D vector encoding properties such as amino acid type, degree of hydrophobicity, and secondary structure role; (5) **Arxiv** [56], a citation network with nodes representing arXiv papers. Each node has a 128-dimensional feature vector aggregated from word embeddings of its title and abstract; (6) **Yelp** [57] a dense network where nodes represent users and services, and edges refer to friendship. Node features are 300-dimensional word embeddings derived by reviews of services; (7) **Products** [56], a large-scale product co-purchase network.

Besides real-world datasets, we also generated a large synthetic dataset **WS-MAG**, extending the core of the real-world MAG240M citation network [58] using a small-world generator [59]. **WS-MAG** enables controlled studies of the impact of graph size $|G|$ for graphs upto billion-scale. Specifically, we vary $|G|$ from 6 million to 1.194 billion while fixing the average node degree to be 2.

Graph Models. We have pre-trained three classes of representative GNNs: GCNs³ [42], GATs³ [43], and GraphSAGE³ [44], for each dataset. For a fair comparison, (1) for all the datasets,

⁴<https://github.com/Yangxin666/GMV>

Dataset	$ V $	$ E $	# node types	# attributes
Cora [52]	2,708	5,429	7	1,433
Elliptic++ [53]	5,156	4,784	3	166
ATLAS [54]	59,148	40,823	2	64
Protein [55]	43,471	162,088	2	3
Arxiv [56]	169K	1.2M	40	128
Yelp [57]	717K	7.9M	100	300
Products [56]	2.4M	61.9M	47	100
WS-MAG [58]	0.398B	0.796B	153	768

TABLE II: Statistics of Datasets (ranked by graph size $|G|$).

we consider the task of node classification, and (2) all methods are applied for the same set of GNNs.

Baselines. We compare our algorithm GVInf against the following baselines: (1) GInf, direct inference using original graph G without views; (2) GVInf-all, a variant of GVInf where GVSel refers to all views without forgetting mechanism; (3) GVInf-ran, a variant of GVInf that replaces GVSel with a random selection of k views from $\mathcal{V}$; (4) GVInf-nomin, a variant of GVInf without view minimization. (5) DSpar [23], state-of-the-art graph compression method, performing edge down-sampling to preserve graph spectral information to accelerate GNN inference. (6) SPGC [24] is a graph compression method that builds a compressed G_c where nodes indistinguishable to GNNs are merged. GNN inference is then directly performed on G_c without decompression while preserving inference accuracy. (7) Channel Pruning [20] identifies channels with high influence on the output to prune irrelevant features, and caches and reuses hidden features of previously visited nodes to reduce redundant computations during inference. (8) GKD [19] transfers topological knowledge from a teacher GNN on the full graph to a student GNN on a smaller graph by aligning Neural Heat Kernels. Comparisons with Channel Pruning and GKD are discussed as a case study in [34].

Evaluation Metrics. We evaluate GVInf, its variants, and baselines with the following measures. (1) Efficiency. (a) Inference Speed-up. We define the speed-up of inference as $\frac{T_{inf}}{T_{vinf}}$, where T_{inf} is the inference time cost of GInf over G , and T_{vinf} represents the inference time cost of view-based and baseline counterparts. (b) Normalized Compression Ratio (ncr). This measures how well (minimized) views reduce the amount of data to be accessed, compared to the size of L -hops subgraph of the test node. It is defined as $ncr = 1 - \frac{|\mathcal{V}_m|}{|G^L|}$; the closer to 1, the better. (2) Effectiveness. We report the accuracy of GInf and GVInf variants. For Yelp where the benchmark task is a multi-class node classification, we report micro $F1$ -score. To ensure a fair evaluation, we report the average performance of 50 different batches of query workload with the same size.

GMVs Generation and Ground-truth. We use domain-specific, *real-world* patterns to generate GMVs. For Atlas, we extract multi-stage cyber attack patterns [54]. For Elliptic++, we use two expert-validated money-laundering patterns, peel chain and spindles [53]. For protein interactions, curated functional groups and interaction networks serve as views [55], reflecting chemically grounded structure. In citation networks such as Cora, Arxiv, and WS-MAG, views correspond to

frequently occurring citation structures [60]. For social recommendation (e.g., Yelp and Product), views denote frequently observed patterns of recommendations or interactions [61].

For each test node $v_i \in V_T$, we extract its L -hop subgraph and extract q subgraphs that match graph patterns via subgraph isomorphism. By default, $|V_T| = 100$ and $q = 5$, resulting 500 GMVs per dataset. (2) For each $v_i \in V_T$, we sample a set of test nodes V'_T with similar embeddings in the “private” fraction $G \setminus G_P$ not visible to inference queries. The ground truth outputs are generated by inference of M over V'_T . This ensures GMVs with memory graphs having no overlap with the private fraction, hence ensuring view-based inference to be tested without prior knowledge of G .

B. Experimental Results

Exp-1: Efficiency of View-based Inference. We evaluate the efficiency of GVInf and its variants under different workloads, model depths, view sizes, and view-selection budgets.

Overall Inference Speed-up. We set $|\mathcal{V}| = 500$, $|W| = 100$, $k = 10$, and use a 3-layer GCN. Table III reports the average speed-up over GInf and the normalized compression ratio (ncr) on **Cora**, **Protein**, **Arxiv**, **Yelp**, and **Products**. GVInf achieves speed-ups from $\times 5$ to $\times 850$, with larger gains on larger graphs. It consistently outperforms its variants, DSpar, and SPGC; GVInf-all and GVInf-ran also outperform GVInf-nomin, confirming the benefits of minimization and selection.

Varying Workload Size $|W|$. With $|\mathcal{V}| = 500$, $k = 10$, and a 3-layer GCN, we vary $|W|$ from 50 to 200. Fig. 9(a) shows that all methods scale linearly with $|W|$. GVInf has the lowest inference cost, whereas GVInf-nomin has the highest, consistent with Table III.

Varying Number of Layers L . With $|\mathcal{V}| = 500$, $|W| = 100$, and $k = 10$, we vary L from 2 to 4. As shown in Fig. 9(b), the inference cost grows approximately linearly with L , including the once-for-all overhead of extending virtual GMVs, consistent with prior GNN inference analysis [62].

Varying Number of Views $|\mathcal{V}|$. With $|W| = 100$, $k = 10$, and a 3-layer GCN, we vary $|\mathcal{V}|$ from 50 to 5,000. Fig. 9(c) shows that inference cost remains largely insensitive to $|\mathcal{V}|$, because reconstruction depends mainly on the selected views and benefits from efficient view selection.

Varying k . On **Products**, we set $|W| = 100$, $|\mathcal{V}| = 500$, and use a 3-layer GCN while varying k from 5 to 20. In Fig. 9(d), the costs of GVInf, GVInf-all, and GVInf-nomin increase initially and then stabilize as the number of useful selected views saturates near $k = 10$. By contrast, GVInf-ran grows linearly because it always selects exactly k views.

Exp-2: Accuracy of View-based Inference. Table IV compares $F1$ scores under the same default setting. GVInf, GVInf-all, and GVInf-nomin remain close to direct inference and outperform DSpar and SPGC; random view selection is consistently less accurate because of poor coverage.

	Cora	Proteins	Arxiv	Yelp	Products
GVInf	5.35 (0.78)	41.85 (0.76)	80.19 (0.81)	138.82 (0.82)	849.98 (0.89)
GVInf-all	4.57 (0.74)	37.48 (0.73)	68.65 (0.78)	114.13 (0.78)	694.81 (0.87)
GVInf-ran	4.66 (0.74)	38.71 (0.74)	64.68 (0.77)	125.41 (0.80)	739.29 (0.88)
GVInf-nomin	2.62 (0.54)	21.53 (0.54)	36.39 (0.59)	61.62 (0.59)	291.01 (0.69)
SPGC	2.23 (0.46)	19.62 (0.49)	31.14 (0.52)	54.67 (0.54)	175.78 (0.49)
Dspar	1.57 (0.24)	15.93 (0.37)	23.31 (0.36)	46.51 (0.46)	124.18 (0.28)

TABLE III: Inference Speed-up and Normalized Compression Ratio ncr. ncr are marked in parentheses. Best performance is highlighted in bold. Inference speed-up is computed relative to Glnf. ncr is computed relative to $G^L(v_t)$ of test node v_t .

	Cora	Protein	Arxiv	Yelp	Products
GVInf	0.7966 $\pm$ 0.0767	0.7356 $\pm$ 0.0741	0.7148 $\pm$ 0.0653	0.6514 $\pm$ 0.0712	0.8031 $\pm$ 0.0693
GVInf-all	0.8055 $\pm$ 0.0604	0.7323 $\pm$ 0.0684	0.7092 $\pm$ 0.0614	0.6581 $\pm$ 0.0679	0.8017 $\pm$ 0.0637
GVInf-ran	0.6722 $\pm$ 0.0831	0.6256 $\pm$ 0.0815	0.6448 $\pm$ 0.0741	0.5531 $\pm$ 0.0786	0.6879 $\pm$ 0.0752
GVInf-nomin	0.7964 $\pm$ 0.0735	0.7302 $\pm$ 0.0726	0.7153 $\pm$ 0.0656	0.6523 $\pm$ 0.0708	0.7985 $\pm$ 0.0611
SPGC	0.7784 $\pm$ 0.0723	0.7275 $\pm$ 0.0741	0.7029 $\pm$ 0.0709	0.6342 $\pm$ 0.0796	0.7791 $\pm$ 0.0764
DSpar	0.7542 $\pm$ 0.0794	0.7165 $\pm$ 0.0783	0.6875 $\pm$ 0.0679	0.6124 $\pm$ 0.0773	0.7567 $\pm$ 0.0786
Glnf	0.7971 $\pm$ 0.0121	0.7387 $\pm$ 0.0147	0.7175 $\pm$ 0.0103	0.6592 $\pm$ 0.0194	0.8017 $\pm$ 0.0258

TABLE IV: Inference Accuracy: Overall Comparison. Best performance is highlighted in bold.

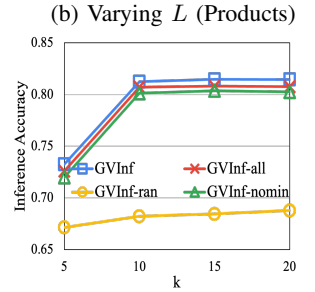
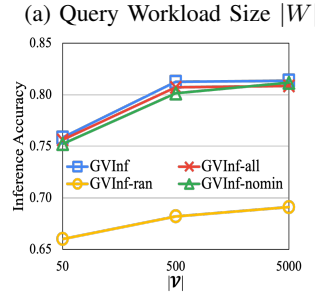
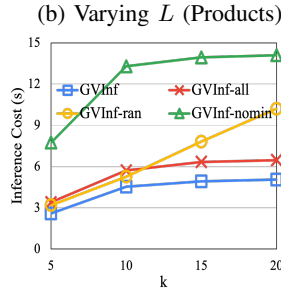
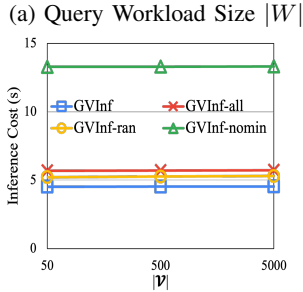
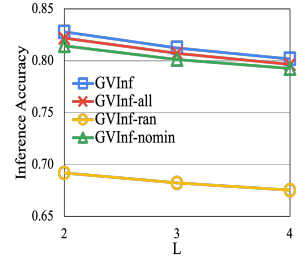
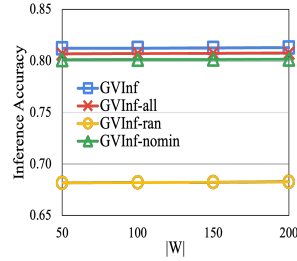
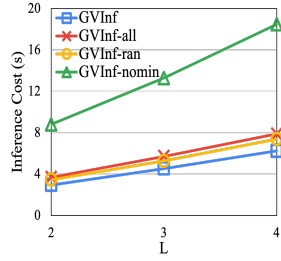
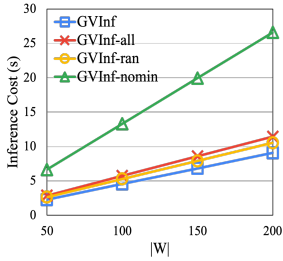


Fig. 9: Inference Costs of View-based Inference.

Sensitivity analysis. On **Products**, we use $|W| = 100$, $|\mathcal{V}| = 500$, $k = 10$, and a 3-layer GCN unless otherwise stated. (1) *Workload size and depth.* Varying $|W|$ from 50 to 200 has little effect on the accuracy of GVInf, GVInf-nomin, and GVInf-all, whereas GVInf-ran remains lower due to poor random-view coverage (Fig. 10(a)). Increasing L from 2 to 4 reduces accuracy for all methods, consistent with GCN oversmoothing [63] (Fig. 10(b)). (2) *Number of views and selection budget.* Accuracy improves as $|\mathcal{V}|$ grows from 50 to 5,000, but saturates beyond $|\mathcal{V}| = 500$ because the top- k views already cover most queries (Fig. 10(c)). Similarly,

increasing k from 5 to 20 improves GVInf, GVInf-all, and GVInf-nomin until coverage saturates near $k = 10$; GVInf-ran gains only slightly because random selection adds limited coverage (Fig. 10(d)).

Exp-3: Inference test for billion-scale G . Fixing $|Q|$ being 100, $|\mathcal{V}|$ being 500, k being 10, GNNs class being a 3-layers GAT, utilizing the simulated graph dataset **WS-MAG**, we vary the size of graph $|G|$ from 6 million to 1.194 billion. (1) The inference cost of SPGC, DSpar, and Glnf significantly increases (Fig. 11(a)). SPGC induces smaller inference cost than DSpar but still more costly than GVInf and its variants.

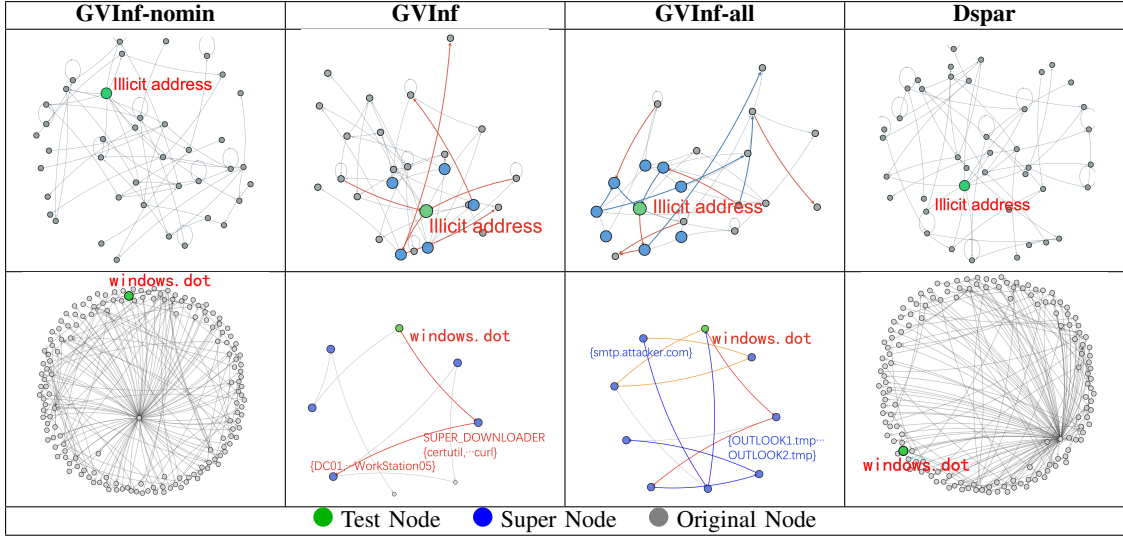


TABLE V: Case Study: Bitcoin Laundering and Document Exploit.

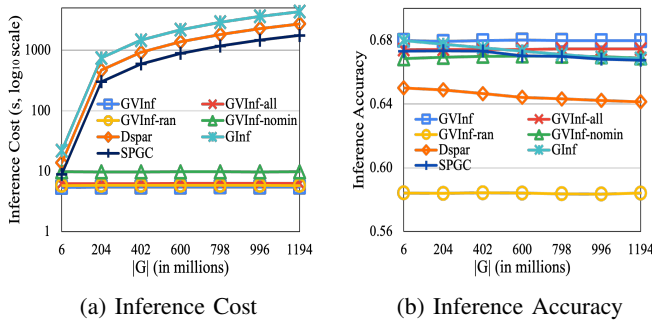


Fig. 11: The Impact of $|G|$ on View based graph inference.

By contrast, the inference cost of GVInf and its variants remain stable regardless of the graph size, as their inference costs are determined by the size of GMVs $|\mathcal{V}|$, which is a relatively small constant. (2) The accuracy of Glnf and its variants remain stable (Fig. 11(b)). Interestingly, the accuracy of Dspar, SPGC and Glnf slightly drops. We found that this is due to the output quality of view-based inference also only depends on memory graphs, while Dspar, SPGC, and Glnf could suffer accuracy loss due to noisy messages and over-smoothing in large graphs [23], [63].

Exp-4: Ablation Analysis. We also conducted ablation analysis to evaluate the effectiveness of selection, minimization and forgetting mechanism to the accuracy, inference cost, and memory, respectively. While each contributed positively to all the measures, GVSel has the most significant contribution to accuracy (14.17% loss if removed); and GVMin benefits most in reducing memory and inference cost (details in [34]).

Summary. View-based graph inference improves direct inference over million-billion scale G by 488 times on average, accessing only 15.7% of original data, with on average only 0.46% loss of accuracy. These suggest promising application of GMVs for large-scale graph analysis.

Exp-5: Case Studies. We showcase GVInf in Bitcoin money-laundering detection [53] and cybersecurity analysis [54].

Case Study I: Bitcoin Money Laundering Detection. GNN classifiers identify illicit accounts in large Bitcoin transaction networks. We wrap two published laundering patterns, *Peel Chain* (self-loops) and *Spindle* (disperse-converge) [64], as GMVs (Table V). All GMV-based methods correctly detect illicit accounts, consistent with Glnf. With minimization, GVInf uses at most five nodes, achieves accuracy 0.78, and provides a $\times 7$ speed-up over Glnf. GVInf-all accesses larger views ($\text{ncr} = 0.78$, accuracy 0.81), whereas GVInf-nomin uses the most view nodes. By contrast, Dspar requires full-network access to learn a sparse structure.

Case Study II: Attack Pattern Detection. Cybersecurity analysts use GNN-based link prediction over system-log graphs to detect attacks involving malicious file access [54]. We wrap three validated patterns as GMVs: $\mathcal{V}_1$ (*Email Delivery*), $\mathcal{V}_2$ (*Exploitation*), and $\mathcal{V}_3$ (*Propagation*); representative paths are listed in Table V. While GVInf-nomin explores all three views over the full 3-hop subnetwork, GVInf merges equivalent handlers such as $\{\text{certutil}, \text{wget}, \text{curl}\}$ into SUPER_DOWNLOADER. It yields five node classes ($\text{ncr} = 0.93$, accuracy 0.72), incurs the lowest inference cost, and achieves a $\times 140$ speed-up over Glnf.

VIII. CONCLUSION

We presented graph model views as a data-engineering abstraction for serving reusable graph-model computations. A GMV couples a memory graph with historical model outputs and the metadata needed for matching and layer-wise message reconstruction. We introduced a sufficient condition for approximating graph inference with certifiable error bound, and provide model class-specific analysis for mainstream GNNs. We developed algorithms for view-based graph inference, as well as view selection, minimization, and workload processing. The existing experiments show $5\times\text{--}850\times$ inference speedups while accessing substantially smaller graph representations and incurring small accuracy loss on the evaluated tasks. A future topic is to deploy GMVs to support graph analysis in federated deployment.

REFERENCES

- [1] P. Veličković, “Everything is connected: Graph neural networks,” *Current Opinion in Structural Biology*, vol. 79, p. 102538, 2023.
- [2] M. Bursch, J.-M. Mewes, A. Hansen, and S. Grimme, “Best-practice dft protocols for basic molecular computational chemistry,” *Angewandte Chemie*, vol. 134, no. 42, p. e202205735, 2022.
- [3] Y. Shee, T.-L. Yeh, J.-Y. Hsiao, A. Yang, Y.-C. Lin, and M.-H. Hsieh, “Quantum simulation of preferred tautomeric state prediction,” *npj Quantum Information*, vol. 9, no. 1, p. 102, 2023.
- [4] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, “The graph neural network model,” *IEEE transactions on neural networks*, vol. 20, no. 1, 2008.
- [5] J. Pei, R. C. Fernandez, and X. Yu, “Data and ai model markets: Opportunities for data and model sharing, discovery, and integration,” *Proceedings of the VLDB Endowment*, vol. 16, no. 12, 2023.
- [6] X. Ma, J. Wu, S. Xue, J. Yang, C. Zhou, Q. Z. Sheng, H. Xiong, and L. Akoglu, “A comprehensive survey on graph anomaly detection with deep learning,” *ICDE*, 2021.
- [7] H. F. AI, “Hugging Face – The AI community building the future.” 2025. [Online]. Available: <https://huggingface.co/>
- [8] T. Hanser, “Federated learning for molecular discovery,” *Current Opinion in Structural Biology*, vol. 79, p. 102545, 2023.
- [9] H. E. Pence and A. Williams, “Chemspider: an online chemical information resource,” 2010.
- [10] P. J. Linstrom and W. G. Mallard, “The nist chemistry webbook: A chemical data resource on the internet,” *Journal of Chemical & Engineering Data*, vol. 46, no. 5, pp. 1059–1063, 2001.
- [11] A. Banerjee, S. Kar, K. Roy, G. Patlewicz, N. Charest, E. Benfenati, and M. T. Cronin, “Molecular similarity in chemical informatics and predictive toxicity modeling: from quantitative read-across (q-ra) to quantitative read-across structure–activity relationship (q-rasar) with the application of machine learning,” *Critical Reviews in Toxicology*, vol. 54, no. 9, 2024.
- [12] W. Fan, X. Wang, and Y. Wu, “Answering graph pattern queries using views,” in *ICDE*, 2014.
- [13] J. Li, Y. Cao, and X. Liu, “Approximating graph pattern queries using views,” in *CIKM*, 2016.
- [14] Q. Song, Y. Wu, P. Lin, L. X. Dong, and H. Sun, “Mining summaries for knowledge graph search,” *ICDE*, 2018.
- [15] C. Zhang, J. Lu, Q. Guo, X. Zhang, X. Han, and M. Zhou, “Automatic view selection in graph databases,” in *Proceedings of the 33rd International Conference on Scientific and Statistical Database Management*, 2021, pp. 197–202.
- [16] F. Frasca, E. Rossi, D. Eynard, B. Chamberlain, M. Bronstein, and F. Monti, “Sign: Scalable inception graph neural networks,” *arXiv preprint arXiv:2004.11198*, 2020.
- [17] A. Bojchevski, J. Gasteiger, B. Perozzi, A. Kapoor, M. Blais, B. Rózemberczki, M. Lukasik, and S. Günnemann, “Scaling graph neural networks with approximate pagerank,” in *SIGKDD*, 2020.
- [18] N. Liao, D. Mo, S. Luo, X. Li, and P. Yin, “Scara: scalable graph neural networks with feature-oriented optimization,” *arXiv preprint arXiv:2207.09179*, 2022.
- [19] C. Yang, Q. Wu, and J. Yan, “Geometric knowledge distillation: Topology compression for graph neural networks,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 29 761–29 775, 2022.
- [20] H. Zhou, A. Srivastava, H. Zeng, R. Kannan, and V. Prasanna, “Accelerating large scale real-time gnn inference using channel pruning,” *arXiv preprint arXiv:2105.04528*, 2021.
- [21] C. Zheng, B. Zong, W. Cheng, D. Song, J. Ni, W. Yu, H. Chen, and W. Wang, “Robust graph representation learning via neural sparsification,” in *PMLR*, 2020.
- [22] D. Li, T. Yang, L. Du, Z. He, and L. Jiang, “Adaptivegcn: Efficient gcn through adaptively sparsifying graphs,” in *CIKM*, 2021, pp. 3206–3210.
- [23] Z. Liu, K. Zhou, Z. Jiang, L. R. Chen, S.-H. Choi, and X. Hu, “Dspar: An embarrassingly simple strategy for efficient gnn training and inference via degree-based sparsification,” *TMLR*, 2023.
- [24] Y. Fan, H. Che, and Y. Wu, “Inference-friendly graph compression for graph neural networks,” *Proc. VLDB Endow.*, vol. 18, no. 9, 2025.
- [25] W. Huang, G. Wan, M. Ye, and B. Du, “Federated graph semantic and structural learning,” *arXiv preprint arXiv:2406.18937*, 2024.
- [26] Y. Shao, H. Li, X. Gu, H. Yin, Y. Li, X. Miao, W. Zhang, B. Cui, and L. Chen, “Distributed graph neural network training: A survey,” *ACM Computing Surveys*, vol. 56, no. 8, pp. 1–39, 2024.
- [27] M. Besta and T. Hoefler, “Parallel and distributed graph neural networks: An in-depth concurrency analysis,” *PAMI*, 2024.
- [28] Z. Cai, X. Yan, Y. Wu, K. Ma, J. Cheng, and F. Yu, “Dgcl: An efficient communication library for distributed gnn training,” in *Proceedings of the Sixteenth European Conference on Computer Systems*, 2021, pp. 130–144.
- [29] T. Kaler, N. Stathas, A. Ouyang, A.-S. Iliopoulos, T. Schardl, C. E. Leiserson, and J. Chen, “Accelerating training and inference of graph neural networks with fast sampling and pipelining,” *MLSys*, 2022.
- [30] D. Zheng, X. Song, C. Yang, D. LaSalle, and G. Karypis, “Distributed hybrid cpu and gpu training for graph neural networks on billion-scale heterogeneous graphs,” in *SIGKDD*, 2022.
- [31] T. Liu, Y. Chen, D. Li, C. Wu, Y. Zhu, J. He, Y. Peng, H. Chen, H. Chen, and C. Guo, “{BGL}:{GPU-Efficient}{GNN} training by optimizing graph data {I/O} and preprocessing,” in *USENIX*, 2023.
- [32] M. Wang, D. Zheng, Z. Ye, Q. Gan, M. Li, X. Song, J. Zhou, C. Ma, L. Yu, Y. Gai *et al.*, “Deep graph library: A graph-centric, highly-performant package for graph neural networks,” *arXiv preprint arXiv:1909.01315*, 2019.
- [33] M. Fey and J. E. Lenssen, “Fast graph representation learning with pytorch geometric,” *arXiv preprint arXiv:1903.02428*, 2019.
- [34] “Full version,” 2026. [Online]. Available: https://github.com/Yangxin666/GMV/blob/main/Approximating_Graph_Inference_by_Serving_Graph_Model_Outputs_as_Views.pdf
- [35] M. Gardner, J. Grus, M. Neumann, O. Tafjord, P. Dasigi, N. Liu, M. Peters, M. Schmitz, and L. Zettlemoyer, “Allennlp: A deep semantic natural language processing platform,” *arXiv preprint arXiv:1803.07640*, 2018.
- [36] P. Barceló, E. V. Kostylev, M. Monet, J. Pérez, J. L. Reutter, and J.-P. Silva, “The expressive power of graph neural networks as a query language,” *SIGMOD Record*, 2020.
- [37] F. Geerts, “A query language perspective on graph learning,” in *PODS*, 2023.
- [38] M. Chen, Z. Wei, B. Ding, Y. Li, Y. Yuan, X. Du, and J.-R. Wen, “Scalable graph neural networks via bidirectional propagation,” *NeurIPS*, 2020.
- [39] S. Abriola, P. Barceló, D. Figueira, and S. Figueira, “Bisimulations on data graphs,” *JAIR*, 2018.
- [40] J. Martens, J. F. Groote, L. van den Haak, P. Hijma, and A. Wijs, “A linear parallel algorithm to compute bisimulation and relational coarsest partitions,” in *International Conference on Formal Aspects of Component Software*, 2021, pp. 115–133.
- [41] J. F. Groote, D. N. Jansen, J. J. A. Keiren, and A. J. Wijs, “An o(mlogn) algorithm for computing stuttering equivalence and branching bisimulation,” *ACM Trans. Comput. Logic*, vol. 18, no. 2, 2017.
- [42] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” *arXiv preprint arXiv:1609.02907*, 2016.
- [43] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, Y. Bengio *et al.*, “Graph attention networks,” *stat*, 2017.
- [44] W. Hamilton, Z. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” *NeurIPS*, 2017.
- [45] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, “How powerful are graph neural networks?” *arXiv preprint arXiv:1810.00826*, 2018.
- [46] OpenML, “Openml: A worldwide machine learning lab,” 2025. [Online]. Available: <https://openml.org/>
- [47] M. R. Henzinger, T. A. Henzinger, and P. W. Kopke, “Computing simulations on finite and infinite graphs,” in *FOCS*, 1995.
- [48] J. Höfberg, A. Maletti, and J. May, “Bisimulation minimisation for weighted tree automata,” in *DLT*, 2007.
- [49] A. Bouali and R. De Simone, “Symbolic bisimulation minimisation,” in *International Conference on Computer Aided Verification*, 1992, pp. 96–108.
- [50] S. S. Bhowmick, S. A. Chen, and D. Srivastava, “Cognitive psychology meets data management: State of the art and future directions,” in *SIGMOD*, 2024.
- [51] J. M. Murre and J. Dros, “Replication and analysis of ebbinghaus’ forgetting curve,” *PloS one*, vol. 10, no. 7, p. e0120644, 2015.
- [52] A. K. McCallum, K. Nigam, J. Rennie, and K. Seymore, “Automating the construction of internet portals with machine learning,” *Information Retrieval*, 2000.
- [53] Y. Elmougy and L. Liu, “Demystifying fraudulent transactions and illicit nodes in the bitcoin network for financial forensics,” in *SIGKDD*, 2023, pp. 3979–3990.

- [54] A. Alsaheel, Y. Nan, S. Ma, L. Yu, G. Walkup, Z. B. Celik, X. Zhang, and D. Xu, “{ATLAS}: A sequence-based learning approach for attack investigation,” in *USENIX*, 2021.
- [55] G. Chen, L. Hou, Z. Li, B. Xie, and Y. Liu, “A new strategy for cas protein recognition based on graph neural networks and smiles encoding,” *Scientific Reports*, vol. 15, no. 1, p. 15236, 2025.
- [56] W. Hu, M. Fey, M. Zitnik, Y. Dong, H. Ren, B. Liu, M. Catasta, and J. Leskovec, “Open graph benchmark: Datasets for machine learning on graphs,” *NeurIPS*, 2020.
- [57] H. Zeng, H. Zhou, A. Srivastava, R. Kannan, and V. Prasanna, “Graphsaint: Graph sampling based inductive learning method,” *arXiv preprint arXiv:1907.04931*, 2019.
- [58] W. Hu, M. Fey, H. Ren, M. Nakata, Y. Dong, and J. Leskovec, “Ogb-lsc: A large-scale challenge for machine learning on graphs,” *NeurIPS*, 2021.
- [59] D. J. Watts and S. H. Strogatz, “Collective dynamics of ‘small-world’ networks,” *nature*, 1998.
- [60] J. Han and J.-R. Wen, “Mining frequent neighborhood patterns in a large labeled graph,” in *CIKM*, 2013.
- [61] J. Leskovec, A. Singh, and J. Kleinberg, “Patterns of influence in a recommendation network,” in *PAKDD*, 2006.
- [62] Z. Wang, Y. Wang, C. Yuan, R. Gu, and Y. Huang, “Empirical analysis of performance bottlenecks in graph neural network training and inference with gpus,” *Neurocomputing*, vol. 446, pp. 165–191, 2021.
- [63] T. K. Rusch, M. M. Bronstein, and S. Mishra, “A survey on oversmoothing in graph neural networks,” *arXiv preprint arXiv:2303.10993*, 2023.
- [64] G. Kappos, H. Yousaf, R. Stütz, S. Rollet, B. Haslhofer, and S. Meiklejohn, “How to peel a million: Validating and expanding bitcoin clusters,” in *USENIX*, 2022.
- [65] M. Fey and J. E. Lenssen, “Fast graph representation learning with PyTorch Geometric,” in *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.
- [66] W.-L. Chiang, X. Liu, S. Si, Y. Li, S. Bengio, and C.-J. Hsieh, “Cluster-gcn: An efficient algorithm for training deep and large graph convolutional networks,” in *SIGKDD*, 2019.

IX. APPENDIX

A. Appendix A: Formal Proofs for the GMV Framework

This appendix gives proofs for the theoretical statements in the main paper. We distinguish structural matching from the stronger, model-aware message-preservation condition used for exact inference. All results that rely only on structural coverage are explicitly identified as approximation or selection results.

Proof of Lemma 1. Let $U = V(G^L(v_t))$ and let U' be the nodes in the candidate depth- L neighborhoods of G_V . Consider the complete set of feature-compatible cross-graph pairs $P_0 = \{(u, u') \in U \times U' : X_u^0 = X_{u'}^0\}$. Define a monotone operator F on relations $P \subseteq P_0$: a pair (u, u') belongs to $F(P)$ iff every eligible incoming and outgoing neighbor of u has a feature-compatible counterpart at u' related by P , and conversely, up to the remaining depth budget. Since $F(P) \subseteq P$ and the finite lattice 2^{P_0} has finite descending chains, repeated refinement $P_{i+1} = F(P_i)$ reaches a unique greatest fixed point R^L . This fixed point is precisely the greatest feature-respecting depth- L bisimulation relation.

The relation is a cross-graph relation and therefore is not reflexive by itself. However, after adding identity pairs within each graph and the inverse relation $(R^L)^{-1}$, the generated relation on the disjoint union is reflexive and symmetric. Transitivity follows from composition of bisimulations: if two nodes have matching features and mutually match all depth-bounded transitions through an intermediate node, then their

transitions can be paired through the composed witnesses. Thus the closure induces an equivalence relation.

For complexity, candidate target nodes in G_V are found in $O(|G_V|)$ time. The pair-refinement implementation maintains, for each candidate pair, counters for unmatched incident transitions and deletes a pair when a counter becomes zero. Each pair and its relevant adjacency information is processed a constant number of times, giving the stated $O(|G_V| + |N^L(v'_t)| |G^L(v_t)|)$ bound for the bounded-depth neighborhoods. $\square$

Proof of Theorem 2. For a participating target node v , write its layer- k update as

$$X_v^k = \Phi_k(X_v^{k-1}, \mathcal{M}_v^k; \Theta^k),$$

where $\mathcal{M}_v^k$ is the complete model-specific multiset of incoming messages, including multiplicities and any coefficients used by the update rule. Let v' be its matched view representative and let $\hat{X}_v^k$ denote the value reconstructed from the GMV.

Proof of Lemma 3. Fix a target node v and layer k . Add and subtract the reconstructed contribution with the target coefficient for every covered neighbor:

$$\begin{aligned} m_v^k - \hat{m}_v^k &= \sum_{u \in C_v^k} \hat{a}_{v\pi(u)}^k (X_u^{k-1} - \hat{X}_{\pi(u)}^{k-1}) \\ &\quad + \sum_{u \in C_v^k} (a_{vu}^k - \hat{a}_{v\pi(u)}^k) X_u^{k-1} + \sum_{u \in U_v^k} a_{vu}^k X_u^{k-1}. \end{aligned}$$

The triangle inequality and $\|X_u^{k-1}\| \leq H_u^{k-1}$ give

$$\|m_v^k - \hat{m}_v^k\| \leq \sum_{u \in C_v^k} |\hat{a}_{v\pi(u)}^k| e_u^{k-1} + q_v^k + r_v^k.$$

Applying the $K_{v,k}$ -Lipschitz update yields the node-wise claim. Stacking the inequalities gives $\mathbf{e}^k \preceq D_k \mathbf{e}^{k-1} + \mathbf{b}_k$. Since the input features of matched nodes are either exact or their layer-0 mismatch is included in $\mathbf{e}^0$, repeated substitution yields

$$\mathbf{e}^L \preceq D_L \cdots D_1 \mathbf{e}^0 + \sum_{j=1}^L D_L \cdots D_{j+1} \mathbf{b}_j.$$

The main statement uses $\mathbf{e}^0 = \mathbf{0}$; otherwise the first term remains. Match multiplicities and coefficients are already represented in $\mathcal{T}_V$. Covered and uncovered coefficient mass follow from the target-local match relation, while H_u^{k-1} can be stored as a layer-wise norm summary or replaced by a valid model-wide bound. Hence the right-hand side is computable without knowing the exact target hidden states.

The older scalar recurrence is recovered by taking maximum norms: if $\|D_k\|_\infty \leq K_k B_k$ and $\|\mathbf{b}_k\|_\infty \leq K_k \eta_k$, then $e_k \leq K_k (B_k e_{k-1} + \eta_k)$. The matrix form is no looser and is generally tighter because it retains the location and downstream influence of each mismatch. $\square$

Certified Error Bounds for GNN Variants. We provide a GNN-class specific analysis and introduce complementary

results. We present a detailed specification and analysis for GCN, GAT, GNNs with node update function using mean and sum aggregation.

Corollary 6: *If $\|D_k\| \leq \rho < 1$ and $\|\mathbf{b}_k\| \leq \beta$ for all k , then $\|\mathbf{e}^L\| \leq \beta(1-\rho^L)/(1-\rho) \leq \beta/(1-\rho)$. Thus, for a contractive GNN, approximation error is bounded independently of depth.* $\square$

Corollary 7: *For mean aggregation, let c_v of d_v neighbors be covered, let μ_C and μ_U be the covered and uncovered neighbor means, and let $\hat{m}_v = \mu_C$. Then*

$$\|m_v - \hat{m}_v\| = \left(1 - \frac{c_v}{d_v}\right) \|\mu_U - \mu_C\| \leq \left(1 - \frac{c_v}{d_v}\right) \text{diam}(\mathcal{H}_v).$$

The certificate depends only on degree, covered count, and a neighborhood-dispersion summary. $\square$

Corollary 8: *For a GCN with symmetric normalization, the fresh error caused by uncovered neighbors of v is bounded by*

$$r_v^k \leq \frac{H_{k-1}}{\sqrt{d_v}} \sum_{u \in U_v^k} \frac{1}{\sqrt{d_u}}.$$

At matrix level, with normalized propagation matrices P and $\hat{P}$, a 1-Lipschitz activation gives

$$\|E^k\|_2 \leq \|W_k\|_2 (\|P\|_2 \|E^{k-1}\|_2 + \|P - \hat{P}\|_2 \|\hat{H}^{k-1}\|_2),$$

and $\|P\|_2 \leq 1$. $\square$

Corollary 9: *For GAT, let $\rho_v^{\text{att}} = \sum_{u \in U_v} \alpha_{vu}$ be uncovered attention mass. If covered attention is renormalized and its ℓ_1 coefficient error is at most τ_v , then*

$$\|m_v - \hat{m}_v\| \leq \rho_v^{\text{att}} \text{diam}(\mathcal{H}_v) + \tau_v H_{\max}.$$

Thus attention-mass coverage, rather than raw edge coverage, controls the local error. $\square$

Corollary 10: *Let z and $\hat{z}$ be the full and GMV logit vectors. If $\|z - \hat{z}\|_\infty \leq \epsilon$ and the full-model top-two margin is $\Delta > 2\epsilon$, the predicted class is unchanged. Equivalently, a GMV prediction is certified using only its approximate margin $\hat{\Delta}$ whenever $\hat{\Delta} > 2\epsilon$.* $\square$

Corollary 11: *For fixed sum/mean message-passing GNNs, exact reuse follows when matched nodes have identical layer-0 features and every neighbor equivalence class has the same multiplicity (and mean denominator) as in $G^L(v_t)$.* $\square$

Corollary 12: *For a fixed GCN, exact reuse additionally requires every reconstructed contribution to use the target coefficient $1/\sqrt{\deg(u)\deg(v)}$.* $\square$

We prove $\hat{X}_v^k = X_v^k$ for every participating node by induction on k . For $k = 0$, equality holds by condition (i). Assume equality for all participating nodes through layer $k-1$. By condition (ii), $\mathcal{T}_v$ maps the incoming view messages to

exactly the same multiset $\mathcal{M}_v^k$ used at the target, with identical multiplicities, normalization factors, sampled-neighbor identities, edge weights, or other coefficients required by M . The induction hypothesis makes the message payloads equal, and the model parameters and deterministic update map are unchanged. Hence the reconstructed and target calls to Φ_k have identical arguments, so $\hat{X}_v^k = X_v^k$. Induction through L gives equality at v_t and at every requested participating output. $\square$

Proof of Corollary 11. For a sum aggregator, equality of the feature-labeled neighbor equivalence classes and their multiplicities makes the two incoming multisets identical at layer 1. Theorem 2’s induction then applies at all later layers. For a mean aggregator, preserving the same multiplicities also preserves the denominator, so the averaged message is identical. $\square$

Proof of Corollary 12. A GCN message from u to v at layer k is $X_u^{k-1}/\sqrt{\deg(u)\deg(v)}$ before the common linear map and activation. The stated condition preserves both the matched payload and the target normalization coefficient for each reconstructed contribution. Therefore the assembled pre-activation message is identical, and Theorem 2 applies. $\square$

Proof of Corollary 4. Form the disjoint union of the selected memory graphs while retaining each node’s view identifier and metadata. If their union jointly satisfies Theorem 2, the proof of that theorem applies verbatim because message reconstruction may retrieve contributions from different views before applying the target update. Conversely, if the union lacks a required message element or model-specific coefficient, the corresponding target update receives a different argument for some admissible parameterization; therefore exactness cannot be guaranteed from the views alone. $\square$

Proof of Corollary 6. Taking any monotone vector norm in Lemma 3 gives $\|\mathbf{e}^k\| \leq \rho \|\mathbf{e}^{k-1}\| + \beta$. With $\mathbf{e}^0 = 0$, induction gives the geometric sum $\beta \sum_{i=0}^{L-1} \rho^i = \beta(1-\rho^L)/(1-\rho)$. The final inequality follows from $0 \leq \rho < 1$. $\square$

Proof of Corollary 7. The full mean is

$$m_v = \frac{c_v}{d_v} \mu_C + \frac{d_v - c_v}{d_v} \mu_U.$$

Subtracting $\hat{m}_v = \mu_C$ gives $m_v - \hat{m}_v = ((d_v - c_v)/d_v)(\mu_U - \mu_C)$, proving equality. Both means lie in the convex hull of the neighborhood representations, so their distance is at most the neighborhood diameter. $\square$

Proof of Corollary 8. For symmetric GCN normalization, an uncovered neighbor contributes $X_u^{k-1}/\sqrt{d_u d_v}$. Summing norms and using $\|X_u^{k-1}\| \leq H_{k-1}$ gives the node-local bound. At matrix level,

$$P H^{k-1} W_k - \hat{P} \hat{H}^{k-1} W_k = P(H^{k-1} - \hat{H}^{k-1})W_k + (P - \hat{P})\hat{H}^{k-1}W_k.$$

Submultiplicativity and the 1-Lipschitz activation prove the recurrence. For an undirected graph with the usual self-loop symmetric normalization, P is similar to a row-stochastic

matrix and has spectral norm at most one, yielding the stated non-expansive propagation factor. $\square$

Proof of Corollary 9. Let $\rho = \rho_v^{\text{att}}$. With exact target attention on covered neighbors, write their normalized weighted mean as μ_C and the uncovered weighted mean as μ_U . Then $m_v = (1 - \rho)\mu_C + \rho\mu_U$, while renormalized covered attention gives $\hat{m}_v = \mu_C$. Thus $\|m_v - \hat{m}_v\| = \rho\|\mu_U - \mu_C\| \leq \rho \text{diam}(\mathcal{H}_v)$. If the covered attention vector is approximated, add and subtract the message formed with the exact covered coefficients. The remaining term is at most $\|\alpha_C - \hat{\alpha}_C\|_1 H_{\max} \leq \tau_v H_{\max}$. $\square$

Proof of Corollary 10. Let $c^* = \arg \max_c z_c$. Every logit changes by at most ϵ , so $\hat{z}_{c^*} \geq z_{c^*} - \epsilon$ and $\hat{z}_c \leq z_c + \epsilon$ for $c \neq c^*$. Hence $\hat{z}_{c^*} - \hat{z}_c \geq \Delta - 2\epsilon > 0$, preserving the class. Conversely, if $\hat{c} = \arg \max_c \hat{z}_c$ and its approximate top-two margin is $\hat{\Delta} > 2\epsilon$, then for every $c \neq \hat{c}$, $\hat{z}_{\hat{c}} - z_c \geq \hat{\Delta} - 2\epsilon > 0$, so the full model has the same prediction. $\square$

NP-hardness of view minimization problem. We reduce Set Cover. Let the universe be $S = \{s_1, \dots, s_m\}$ with subsets $S_1, \dots, S_n$ and budget k . Construct a target root v_t with one uniquely featured leaf u_j for each element s_j . Use a two-layer sum GNN whose first-layer messages retain these one-hot leaf features. For each subset S_i , construct a virtual GMV whose root has the same feature as v_t and whose leaves are exactly the uniquely featured nodes corresponding to elements in S_i . A collection of at most k views jointly satisfies the exact message-preservation condition at the root iff the corresponding subsets cover every element of S : each unique target leaf message must occur in at least one selected view, and no other view can supply it. The construction is polynomial. Thus a polynomial algorithm for minimum exact view selection would solve Set Cover, proving NP-hardness.

Approximation guarantee of Algorithm GVSel. Let Ω be the finite set of target message elements represented in the certificate of Lemma 3. Assign each element $x \in \Omega$ a fixed nonnegative weight w_x equal to an upper bound on its local mismatch contribution multiplied by its downstream influence to v_t in the matrix expansion. A view $\mathcal{V}_i$ covers a subset $C_i \subseteq \Omega$. For $A \subseteq \mathcal{V}$, define

$$F(A) = \sum_{x \in \cup_{\mathcal{V}_i \in A} C_i} w_x.$$

Normalization and monotonicity are immediate. For $A \subseteq B$ and a view V , every newly covered element under $B \cup \{V\}$ is also newly covered under $A \cup \{V\}$ or was already covered by B ; therefore $F(A \cup \{V\}) - F(A) \geq F(B \cup \{V\}) - F(B)$, proving submodularity. The standard cardinality-constrained greedy analysis gives $F(A_g) \geq (1 - (1 - 1/k)^k)F(A^*) \geq (1 - 1/e)F(A^*)$.

Let $W = \sum_{x \in \Omega} w_x$ be total certified mismatch mass before selection. Since selecting A removes the covered terms from the certificate, the residual is at most $R(A) = W - F(A)$. Substitution of the greedy guarantee gives $R(A_g) \leq W - (1 - 1/e)F(A^*)$. This is an additive residual-certificate guarantee; it does not assert a multiplicative approximation to the

Notation	Description
$G = (X, A)$	graph G , X : feature matrix, A : adjacency matrix
V_T	test node set
M_v	node update function
AGG	aggregation function in GNN (e.g., $\sum$, Avg, $\ $)
x_v^k	embedding of node v at layer k
$\mathcal{M}, \mathcal{M}^L$	a GNNs model, set of GNNs models with k layers
$Q = (\mathcal{M}, V_T, G)$	an inference query Q over V_T
$G^L(v_i)$	L -hop induced subgraph of G from v_i
$G_{\mathcal{V}}^c$	minimal memory graph (canonical form)
$\mathcal{V} = (G_{\mathcal{V}}, \mathcal{T}_{\mathcal{V}}, Z_{\mathcal{V}})$	a graph model view $\mathcal{V}$, $G_{\mathcal{V}}$: memory graph, $\mathcal{T}_{\mathcal{V}}$: memoization structure, $Z_{\mathcal{V}}$: the result of Q
$R^L, R_{\equiv}^L, R_{\prec}^L$	L -bisimulation relation, inference equivalence relation, inference dominance relation

TABLE VI: Summary of Notations.

unknown true nonlinear output error. Unit weights recover structural coverage. $\square$

Proof of Lemma 5. Assume the exact model-aware dominance relation. For every message role represented by a dominated node v , the dominating node v' has a corresponding role with the same layer-0 feature, neighbor-class multiplicities, and all coefficients in the model signature. Replace every use of v in a reconstruction witness by the corresponding use of v' . At layer 0 the replacement preserves payloads. Inductively, the model-signature agreement and the replacement witnesses preserve the complete incoming message multiset at each layer, so the deterministic update produces the same representation. Therefore removing v cannot invalidate any exact query witness represented by the view. If dominance is only structural, the replacement need not preserve normalization, sampling, edge weights, or attention, which is why the main text treats that case as approximate. $\square$

Summary of Notations. We summarize the main notations in Table VI.

B. Appendix B: Algorithms and Analysis

A Parallel Implementation of GVMin. Given sufficient computational resources (i.e., multi-cores CPUs or GPUs), GVMin can be accelerated by adopting the Par-BCRP Algorithm [40], the first known linear-time method for computing strong bisimulation via relational coarsest partitioning. By distributing up to $n = \max\{|V^L|, |E^L|\}$ parallel processors, GVMin can parallelize the workloads of iterative branching bisimulation partition refinement across all available units. This ensures GVMin to achieve a linear-time complexity of $O(|V^L| + |E^L|)$ given L and k are small constants, making it highly efficient and scalable for large-scale applications.

Forgetting Model. We also introduce a dynamic maintenance strategy, that aim to keep “useful” GMVs for the query stream as long as possible, hence maximize the reusability

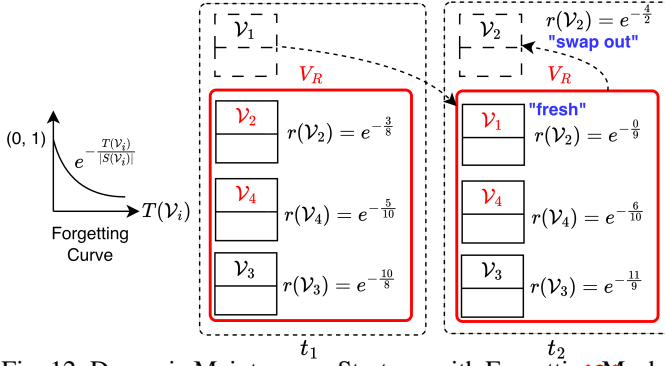


Fig. 12: Dynamic Maintenance Strategy with Forgetting Mechanism. Discarding stale views in $\mathcal{V}$ and serves only the current “fresh” subset $\mathcal{V}_R$.

of GMVs and reduce the selection cost. Our strategy, inspired by established Ebbinghaus’ Forgetting Curve theory [50], [51], poses a forgetting curve model to maintain important memory graphs for GNN M . Following the principle that memory decays exponentially over time, and most forgetting happens soon after learning or inference with a “rate of forgetting” slows down over time (what you remember after some time tends to remain stable longer), We use a view cache to swap out memory graphs that are no longer “fresh” and retain those expected to be needed ($\mathcal{V}_R$), following a forgetting curve expressed by an exponential decay model. Specifically, each view $\mathcal{V}$ is assigned a retention score r :

$$r(\mathcal{V}_i) = e^{-\frac{T_{ep}(\mathcal{V}_i)}{|S(\mathcal{V}_i)|}}$$

where e (the Euler’s number) is a constant 2.71828, $S(\mathcal{V}_i)$ refers to the fraction of $N^L(v)$ covered by the memory graph G_i of $\mathcal{V}_i$, and $T_{ep}(\mathcal{V}_i)$ is the time elapsed since $\mathcal{V}_i$ is referred to for evaluating any inference query. In practice, the view cache is dynamically maintained by the procedure GVSel as top $m * k$ views with highest retention scores where $m \geq 1$. The ranking of retention scores ensure that the candidate views in view cache are always the most relevant views to answering Q among the set of all available views $\mathcal{V}$, so GVMin can potentially cover Q with fewer views. Meanwhile, GVMin can also be accelerated since its time complexity is reduced from $O(k|\mathcal{V}|)$ to $O(k|\mathcal{V}_R|)$.

Example 7: Continuing Example 5, two views $\mathcal{V}_2$ and $\mathcal{V}_4$ are selected at t_1 . At t_1 , $\mathcal{V}_R = \{\mathcal{V}_2, \mathcal{V}_4, \mathcal{V}_3\}$. Assume $k = 2$ and cache size $m = 3$, and the time elapse between t_2 and t_1 is equal to 1. Upon receiving a new inference query Q' at t_2 , forgetting model updates the retention scores for all the views. For the freshly received view $\mathcal{V}_1$, time elapsed $T_{ep}(\mathcal{V}_1) = 0$ and the size of the fraction covered $|S(\mathcal{V}_1)| = 9$, thus its retention score $r(\mathcal{V}_1) = e^{-\frac{0}{9}}$. For $\mathcal{V}_4$, since $t_2 - t_1 = 1$, the time elapsed $T_{ep}(\mathcal{V}_4) = 5 + 1 = 6$ and $|S(\mathcal{V}_4)| = 10$, $r(\mathcal{V}_4) = e^{-\frac{6}{10}}$. For $\mathcal{V}_3$, $T_{ep}(\mathcal{V}_3) = 10 + 1 = 11$ and $|S(\mathcal{V}_3)| = 9$. For $\mathcal{V}_2$, $T_{ep}(\mathcal{V}_2) = 3 + 1 = 4$ and $|S(\mathcal{V}_2)| = 2$. At t_2 , $r(\mathcal{V}_2) = e^{-\frac{4}{2}}$ is smaller than $r(\mathcal{V}_1)$, $r(\mathcal{V}_4)$, and $r(\mathcal{V}_3)$, forgetting model swaps out the view $\mathcal{V}_2$ at t_2 for view $\mathcal{V}_1$. Therefore, at t_2 , $\mathcal{V}_R =$

Algorithm 4 : GMVInf (query workload)

Input: Graph G , a set of views $\mathcal{V}$, a set of test nodes V_T , a GNN $M \in \mathbb{M}^L$ with node update function M_v , query workload $W = \{Q_1, Q_2, \dots, Q_{|V_T|}\}$;

Output: Approximate answer $\mathbf{A}$ to W ;

```

1: set  $\mathbf{A} := \emptyset$  ;
2: /* preprocessing */
3:  $\mathcal{V}_m := \bigcup_{\mathcal{V}_i \in \mathcal{V}} \text{GVMin}(\mathcal{V}_i)$ ;
4: /* workload processing */
5: for  $Q_i \in W$  do
6:   /* select a  $k$ -subset  $\mathcal{V}'_m$  */
7:    $\mathcal{V}'_m := \text{GVSel}(Q_i, \mathcal{V}_m)$ ;
8:   /* view-based inference */
9:   if  $\mathcal{V}'_m \neq \emptyset$  then
10:     $\mathbf{A} := \mathbf{A} \cup \text{GVInf}(N^L(v_i), \mathcal{V}'_m, M^L)$ ;
11: return  $\mathbf{A}$ ;
```

Fig. 13: Algorithm GMVInf.

$\{\mathcal{V}_1, \mathcal{V}_4, \mathcal{V}_3\}$. Since $k = 2$, only two views $\mathcal{V}_1$ and $\mathcal{V}_4$ are selected at t_2 . $\square$

Details of Procedure GVSel. The procedure GVSel first initializes an empty GMV set $\mathcal{V}'$ and an empty coverage set U (line 1). For each candidate view $\mathcal{V}_i \in \mathcal{V}$, GVSel computes $S[\mathcal{V}_i]$ the set of nodes in $N^L(v)$ (line 2). While there are still unselected views that covers uncovered nodes, GVSel conducts the following greedy selection strategy: (1) GVSel updates the uncovered portion of each view by removing nodes already in U (line 4); (2) GVSel selects the view $\mathcal{V}^*$ that covers the largest number of remaining uncovered nodes (line 5); and (3) GVSel adds $\mathcal{V}^*$ to $\mathcal{V}'$ and expand U with the nodes covered by $\mathcal{V}^*$ (line 6-7). This procedure stops when no additional GMVs can be selected.

Query Workload Processing. We present the pseudo-code of the algorithm GMVInf in Fig. 13.

C. Appendix C: Complementary Experimental Study and Discussion

Case Study III: Comparison with Model- and Learning-Based Methods. We further compare GVInf with the state-of-the-art model- and learning- based inference acceleration methods, including CP (Channel Pruning) [20] and GKD (Geometric Knowledge Distillation) [19]. As shown in table VII, GVInf achieves the best trade-off between inference acceleration and inference accuracy across all datasets. For a fair comparison, we set the prune ratio for CP to 50% and reduce the channel size of GKD from 128 to 16 (GKD-16) and 8 (GKD-8) after distillation. On **Cora**, GVInf achieves a $5.35\times$ speed-up with negligible 0.06% drop in accuracy, outperforming CP and all GKD variants. On larger datasets **Arxiv** and **Yelp**, the advantage of GVInf becomes even more pronounced, achieving $80.19\times$ and $138.82\times$ speed-ups while maintaining minimal accuracy drops ($< 1.2\%$). In contrast, CP provides limited speed-ups and incurs significantly higher

accuracy drops (up to 49.94%), while GKD without retraining leads to severe degradation. Although retraining student GNNs (GKD-16 w. Retrain and GKD-8 w. Retrain) alleviate the accuracy loss for GKD to some extent, they introduce substantial computational overhead, yield no inference speed-ups, and still results in higher accuracy drops compared to GVInf. These results demonstrate that GVInf consistently outperforms both CP and GKD and achieves substantially higher inference speed-ups while incurring negligible inference accuracy drops.

Exp-4: Ablation Analysis. Fixing $|W|$ being 500, GNN class being a 3-layers GraphSAGE, $|\mathcal{V}|$ being 500, and k being 10, we conduct ablation studies to evaluate how essential components view selection GVSel, view minimization GVMin, and forgetting mechanism FM affects the inference efficiency (including inference time per query load and memory usage of V_m per test node) and inference accuracy (%). We present results computed on average across all datasets as illustrated in Table VIII.

W vs. W/O GVSel. Removing view selection leads to a substantial 14.17% accuracy drop (from 74.24% to 63.72%), 62.03% increases in memory usage per test node and increases inference time by 31.93% from 5.92 s to 7.81 s per query load. This verifies that GVSel effectively prioritizes informative views and reduces redundant information, leading to improved accuracy and efficiency.

W vs. W/O GVMin. Without GVMin, the inference accuracy drops slightly (from 74.24% to 73.21%) while leading to 135.64% increases in inference time from 5.92 s to 13.95 s per query load and 175.32% increases in memory usage per test node. This indicates that the effectiveness of GVMin serve as a computational optimization tool to accelerate inference and reduce the memory footprints without harming inference accuracy.

W vs. W/O FM. Disabling the forgetting mechanism, the inference accuracy remains nearly unchanged. However, the inference time rises up 7.09% from 5.92 to 6.34 s per query load and memory usage per test node increases by 36.08%. This demonstrates that the FM component primarily functions as a lightweight optimization strategy to accelerate inference by disregarding stale or less relevant cached views while preserving inference accuracy.

Exp-5: Memory Costs. We conduct memory cost analysis on *Arxiv*, *Yelp*, and *Ogbn-Products* datasets. For each dataset, we report the memory costs of the original graph G and the generated 500 views, including the memory cost of all the memory graphs and all the auxiliary structure respectively. We define memory compression rate (mcr) as $mcr = 1 - \frac{|M_c|}{|M|}$. It quantifies the fraction of memory cost that is “reduced”: the larger, the better; The detailed results are presented in Table IX. We conduct the following tests:

(1) **Graph Memory (GM) Test:** We measure the graph memory usage as the graph size in the PyTorch Geometric Graph Data format [65] including node features, node labels, and edge index tensors. Comparing to G , the GM for mem-

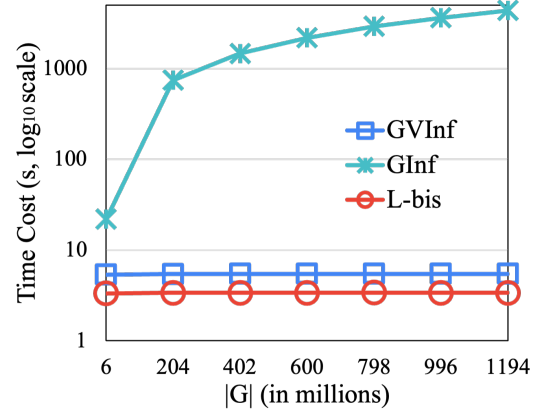


Fig. 14: Time Costs of L -bisimulation, GVInf, and GInf.

ory graphs much smaller, reduced by 75.30%, 84.32%, and 85.76% compared to the original graph G for *Arxiv*, *Yelp*, and *Ogbn-Products* respectively;

(2) **Auxiliary Structure $\mathcal{T}$ Memory (ASM) Test:** We evaluate the memory cost of the auxiliary structure $\mathcal{T}$. We observe the followings: i): On average, the memory cost of $\mathcal{T}$ accounts for only 12.13% of that of G_c on average across all the three datasets; and ii) The combined memory cost of $G_c + \mathcal{T}$ still remains significantly smaller than that of G , with reductions of 72.52%, 83.19%, and 83.54% for *Arxiv*, *Yelp*, and *Ogbn-Products* respectively;

(3) **Peak Memory (PM) Test:** We measure the peak memory usage during the inference process using a 3-layers GCN model, using a hidden size of 32. Compared to inference over the original graph G , inference over the set of views $\mathcal{V}$ reduces the peak memory by 65.97%, 81.61%, and 82.60% for *Arxiv*, *Yelp*, and *Ogbn-Products*. Notably, as the size of original graph increases, the reduction in peak memory becomes more pronounced, reflecting the overall decrease in both graph size and memory footprints.

Exp-6: Time Costs of L -bisimulation. The cost of full GNN inference which is in $O(L|E|F + L|V|F^2)$, with F reaching millions [56], [66]. In contrast, L -bisimulation is in $O(|N_L(v'_t)| |G_L(v_t)|)$ time as proved in Sec. III-C, both are small constants, making it far cheaper. We report the time cost of computing L -bisimulation. Fixing $|Q|$ being 100, $|\mathcal{V}|$ being 500, k being 10, GNNs class being a 3-layers GAT, utilizing the simulated graph dataset **WS-MAG**, we vary the size of graph $|G|$ from 6 million to 1.194 billion to study how graph size affects the time cost of computing L -bisimulation, compared to inference time cost of GVInf and GInf. As $|G|$ increases from 6 million to 1.194 billion, the inference costs of computing L -simulation remains stable regardless of the graph size, as their costs are determined by the size of GMVs $|\mathcal{V}|$ while being significantly faster than GVInf and GInf. On average, it takes 3.39 seconds to compute L -bisimulation over graph with the size of 1.2 billion.

Dataset	CP	GKD-16 w. Retrain	GKD-16 w/o. Retrain	GKD-8 w. Retrain	GKD-8 w/o. Retrain	GVInf
Cora	$1.36\times$ (-2.23%)	$\downarrow 0.01\times$ (-0.12%)	$2.99\times$ (-9.29%)	$\downarrow 0.01\times$ (-0.50%)	$3.36\times$ (-22.80%)	$5.35\times$ (-0.06%)
Arxiv	$1.56\times$ (-14.22%)	$\downarrow 0.01\times$ (-6.14%)	$2.31\times$ (-88.68%)	$\downarrow 0.01\times$ (-8.40%)	$2.65\times$ (-93.33%)	$80.19\times$ (-0.38%)
Yelp	$1.31\times$ (-49.94%)	$\downarrow 0.01\times$ (-2.05%)	$1.73\times$ (-86.36%)	$\downarrow 0.01\times$ (-2.92%)	$1.78\times$ (-96.39%)	$138.82\times$ (-1.18%)

TABLE VII: Comparison of inference speed-up and accuracy loss across CP (Channel Pruning), GKD (Geometric Knowledge Distillation), and GVInf. Both metrics are measured relative to inference on the original graph with pre-trained GNNs, where speed-up is computed using total processing time, including time costs of inference and compression/retraining.

Component	Accuracy (%)	Inf. ($s/ Q $)	Mem. (MB/v)
GVInf	74.24%	5.92	1.58
w/o GVSel	63.72% ($\downarrow$ 14.17%)	7.81 ($\uparrow$ 31.93%)	2.56 ($\uparrow$ 62.03%)
w/o GVMin	73.21% ($\downarrow$ 1.39%)	13.95 ($\uparrow$ 135.64%)	4.35 ($\uparrow$ 175.32%)
w/o FM	74.03% ($\downarrow$ 0.28%)	6.34 ($\uparrow$ 7.09%)	2.15 ($\uparrow$ 36.08%)

TABLE VIII: Ablation study to evaluate key components.

Arxiv	$ V + E $	GM	ASM	PM
G	1,335,586	101.13 MB	–	1.44 GB
$\mathcal{V}$	431,502	24.98 MB	2.81 MB	0.49 GB
mcr	–	75.30%	–	65.97%
Yelp	$ V + E $	GM	ASM	PM
G	14,671,666	1036.03 MB	–	14.41 GB
$\mathcal{V}$	2,742,325	162.51 MB	11.56 MB	2.65 GB
mcr	–	84.32%	–	81.61%
Ogbn-Products	$ V + E $	GM	ASM	PM
G	64,308,169	1887.47 MB	–	30.12 GB
$\mathcal{V}$	11,105,726	268.75 MB	41.78 MB	5.24 GB
mcr	–	85.76%	–	82.60%

TABLE IX: Memory cost comparison between the original graph G and the set of views graph $\mathcal{V}$.